Time Series Analysis

A **time series** is a sequence of measurements from a system that varies in time. Many of the tools we used in previous chapters, like regression, can also be used with time series. But there are additional methods that are particularly useful for this kind of data.

As examples, we'll look at two datasets: renewable electricity generation in the United States from 2001 to 2024, and weather data over the same interval. We will develop methods to decompose a time series into a long-term trend and a repeated seasonal component. We'll use linear regression models to fit and forecast trends. And we'll try out a widely-used model for analyzing time series data, with the formal name "autoregressive integrated moving average" and the easier-to-say acronym ARIMA.

The third edition of *Think Stats* is available now from Bookshop.org and Amazon (those are affiliate links). If you are enjoying the free, online version, consider buying me a coffee.

Click here to run this notebook on Colab.

```
from os.path import basename, exists


def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve

        local, _ = urlretrieve(url, filename)
        print("Downloaded " + local)


download("https://github.com/AllenDowney/ThinkStats/raw/v3/nb/thinkstats.py")
```

```
try:
    import empiricaldist
except ImportError:
    %pip install empiricaldist
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from thinkstats import decorate

plt.rcParams["figure.dpi"] = 300
```

Electricity

As an example of time-series data, we'll use a dataset from the U.S. Energy Information Administration -- it includes total electricity generation per month from renewable sources from 2001 to 2024. Instructions for downloading the data are in the notebook for this chapter.

The following cell downloads the data, which I downloaded September 17, 2024 from https://www.eia.gov/electricity/data/browser/.

```
filename = "Net_generation_for_all_sectors.csv"
download("https://github.com/AllenDowney/ThinkStats/raw/v3/data/" + filename)
```

After loading the data, we have to make some transformations to get it into a format that's easy to work with.

```
elec = (
    pd.read_csv("Net_generation_for_all_sectors.csv", skiprows=4)
    .drop(columns=["units", "source key"])
    .set_index("description")
    .replace("--", np.nan)
    .transpose()
    .astype(float)
)
```

In the reformatted dataset, each column is a sequence of monthly totals in gigawatt-hours (GWh). Here are the column labels, showing the different sources of electricity, or "sectors".

```
elec.columns
```

```
Index(['Net generation for all sectors', 'United States',
       'United States : all fuels (utility-scale)', 'United States : nuclear',
       'United States : conventional hydroelectric',
       'United States : other renewables', 'United States : wind',
       'United States : all utility-scale solar', 'United States : geothermal',
       'United States : biomass',
       'United States : hydro-electric pumped storage',
       'United States : all solar',
       'United States : small-scale solar photovoltaic'],
      dtype='object', name='description')
```

The labels in the index are strings indicating months and years -- here are the first 12.

```
elec.index[:12]
```

```
Index(['Jan 2001', 'Feb 2001', 'Mar 2001', 'Apr 2001', 'May 2001', 'Jun 2001',
       'Jul 2001', 'Aug 2001', 'Sep 2001', 'Oct 2001', 'Nov 2001', 'Dec 2001'],
      dtype='object')
```

It will be easier to work with this data if we replace these strings with Pandas `Timestamp` objects. We can use the `date_range` function to generate a sequence of `Timestamp` objects, starting in January 2001 with the frequency code `"ME"`, which stands for "month end", so it fills in the last day of each month.

```
elec.index = pd.date_range(start="2001-01", periods=len(elec), freq="ME")
elec.index[:6]
```

```
DatetimeIndex(['2001-01-31', '2001-02-28', '2001-03-31', '2001-04-30',
               '2001-05-31', '2001-06-30'],
              dtype='datetime64[ns]', freq='ME')
```

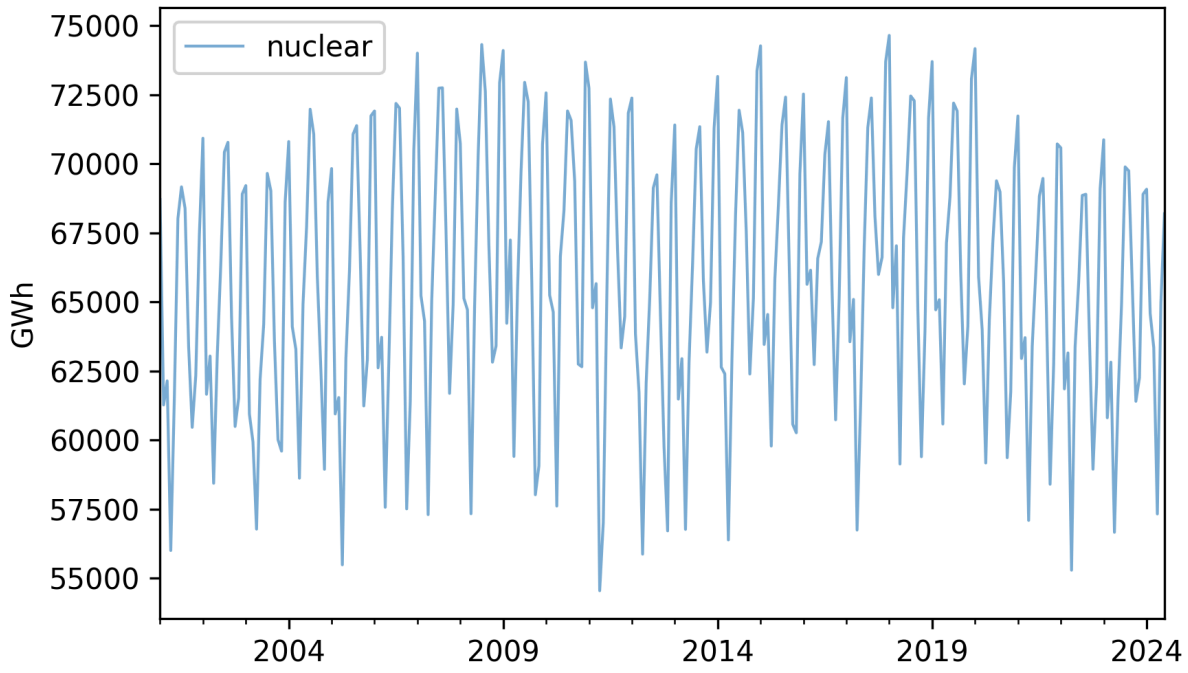
Now the index is a `DataTimeIndex` with the data type `datetime64[ns]`, which is defined in NumPy -- `64` means each label uses 64 bits, and `ns` means it has nanosecond precision.

Decomposition

As a first example, we'll look at how electricity generation from nuclear reactors changed over the interval from January 2001 to June 2024, and we'll decompose the time series into a long-term trend and a periodic component. Here are monthly totals of electricity generation from nuclear reactors in the United States.

```
actual_options = dict(color="C0", lw=1, alpha=0.6)
trend_options = dict(color="C1", alpha=0.6)
pred_options = dict(color="C2", alpha=0.6, ls=':')
model_options = dict(color="gray", alpha=0.6, ls='--')
```

```
nuclear = elec["United States : nuclear"]
nuclear.plot(label="nuclear", **actual_options)

decorate(ylabel="GWh")
```



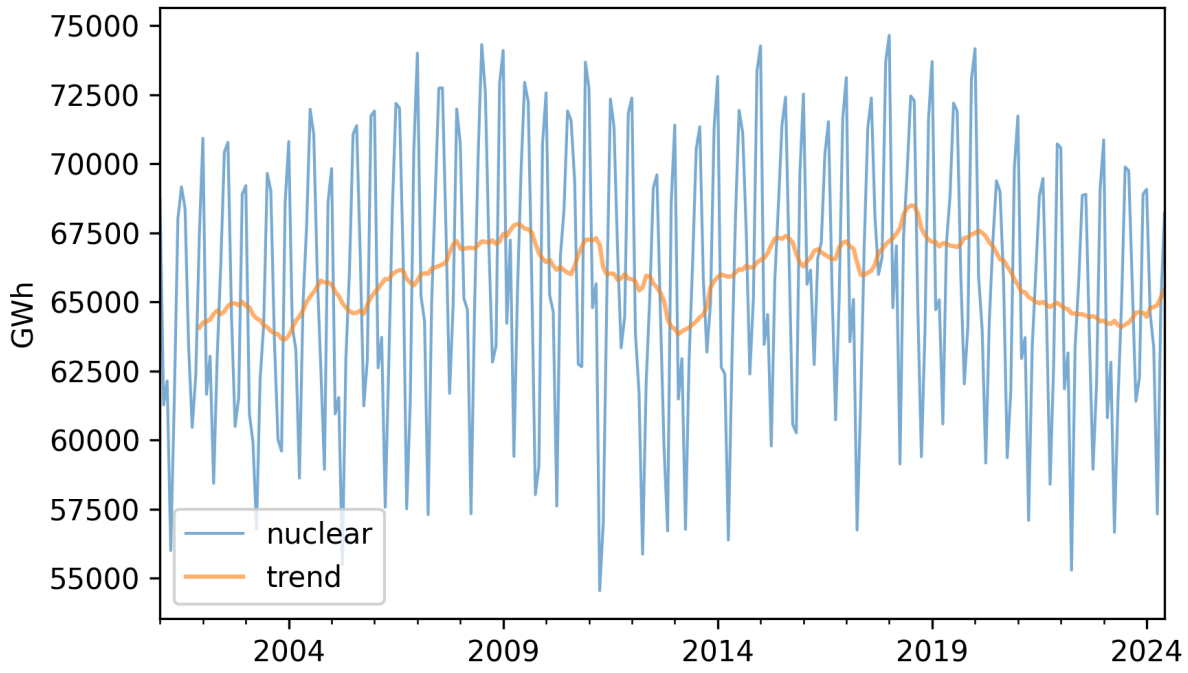
It looks like there are some increases and decreases, but they are hard to see clearly because there are large variations from month to month. To see the long-term trend more clearly, we can use the `rolling` and `mean` methods to compute a **moving average**.

```
trend = nuclear.rolling(window=12).mean()
```

The `window=12` argument selects overlapping intervals of 12 months, so the first interval contains 12 measurements starting with the first, the second interval contains 12 measurements starting with the second, and so on. For each interval, we compute the mean production.

Here's what the results look like, along with the original data.

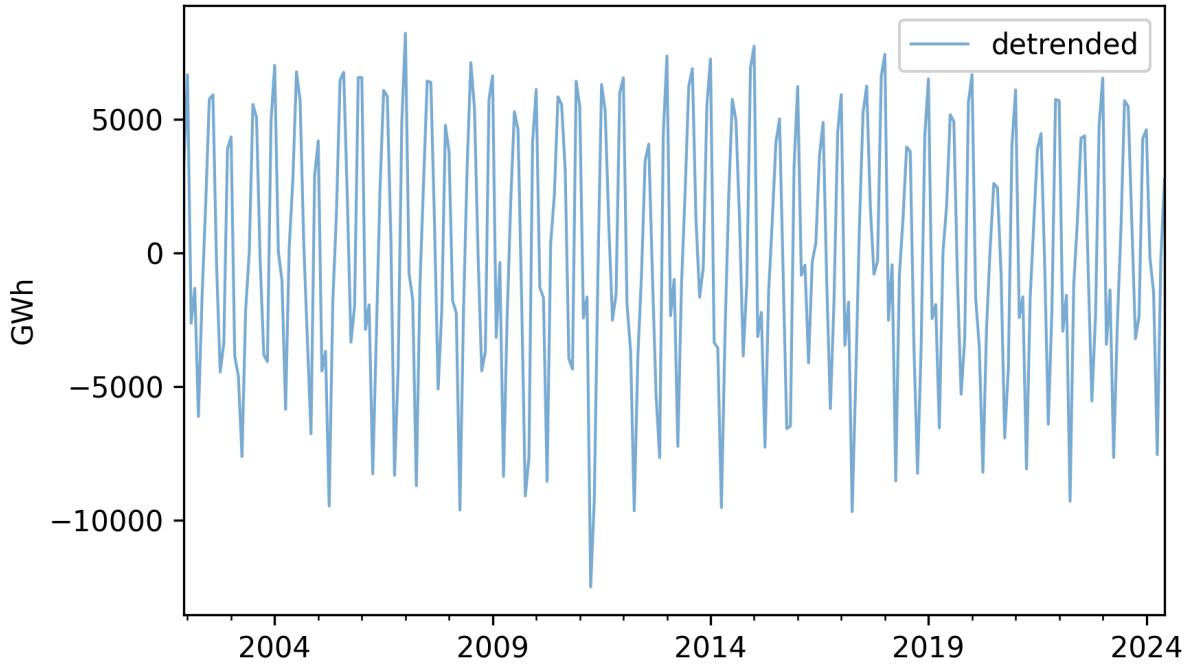
```
nuclear.plot(label="nuclear", **actual_options)
trend.plot(label="trend", **trend_options)
decorate(ylabel="GWh")
```



The trend is still quite variable. We could smooth it more by using a longer window, but we'll stick with the 12-month window for now.

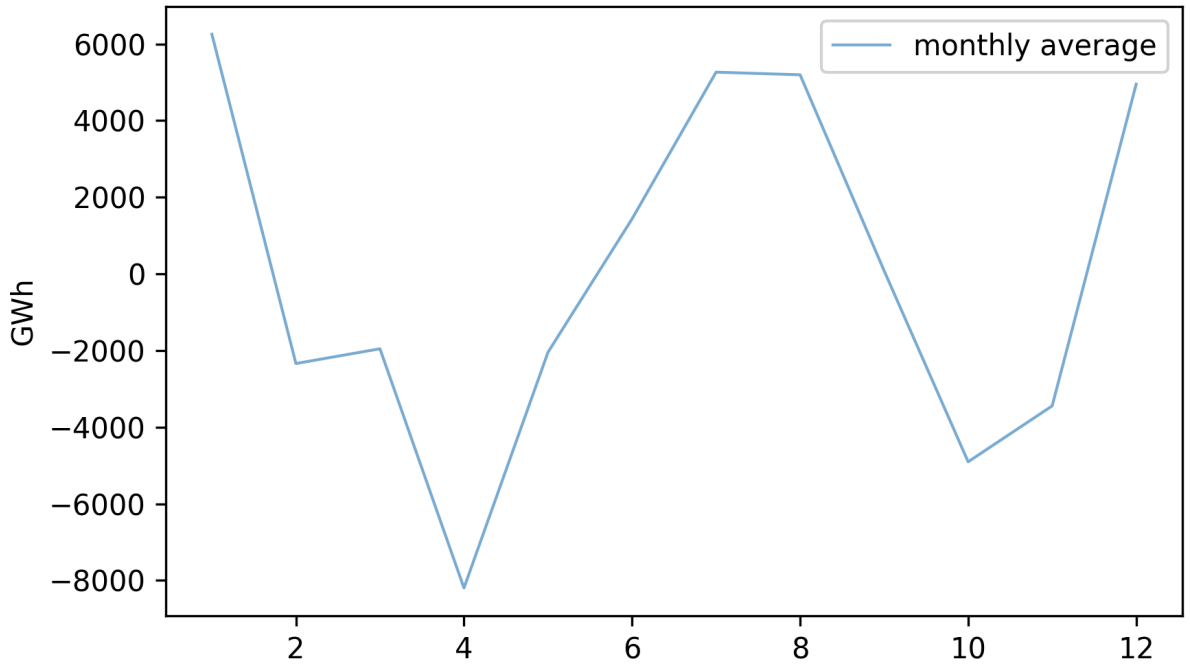
If we subtract the trend from the original data, the result is a "detrended" time series, which means that the long-term mean is close to constant. Here's what it looks like.

```
detrended = (nuclear - trend).dropna()
detrended.plot(label="detrended", **actual_options)
decorate(ylabel="GWh")
```



It seems like there is a repeating annual pattern, which makes sense because demand for electricity varies from one season to another, as it is used to generate heat in the winter and run air conditioning in the summer. To describe this annual pattern we can select the month part of the `datetime` objects in the index, group the data by month, and compute average production. Here's what the monthly averages look like.

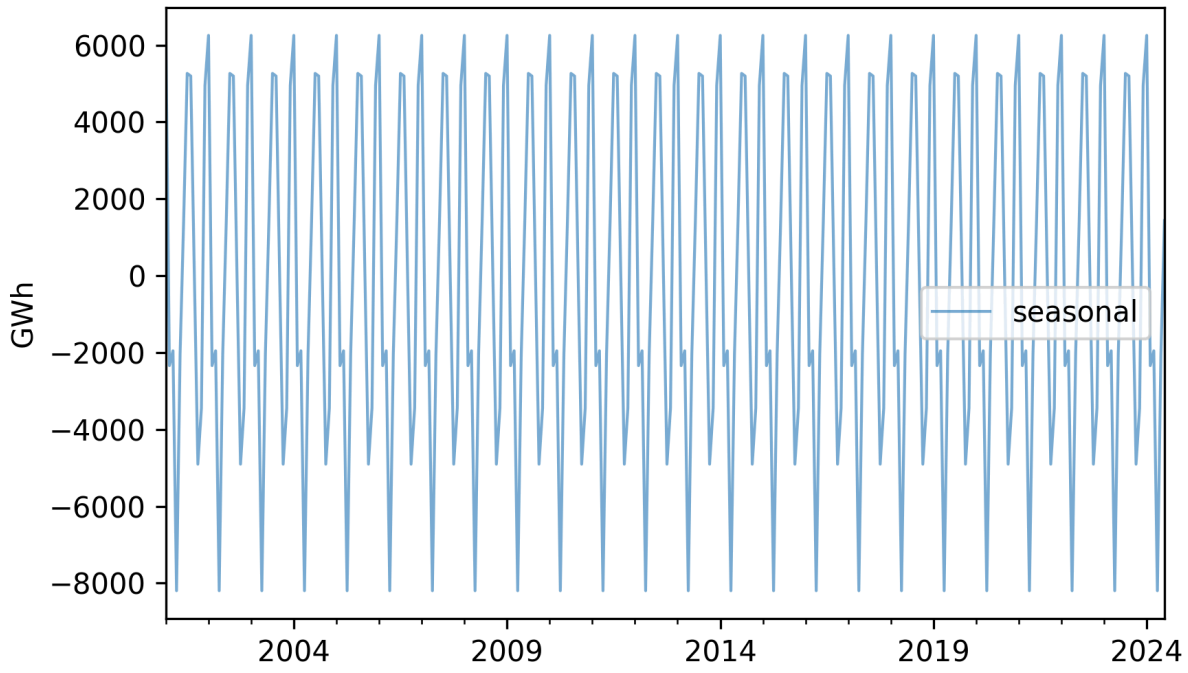
```
monthly_averages = detrended.groupby(detrended.index.month).mean()
monthly_averages.plot(label="monthly average", **actual_options)
decorate(ylabel="GWh")
```



On the x-axis, month 1 is January and month 12 is December. Electricity production is highest during the coldest and warmest months, and lowest during April and October.

We can use `monthly_averages` to construct the seasonal component of the data, which is a series the same length as `nuclear`, where the element for each month is the average for that month. Here's what it looks like.

```
seasonal = monthly_averages[nuclear.index.month]
seasonal.index = nuclear.index
seasonal.plot(label="seasonal", **actual_options)
decorate(ylabel="GWh")
```



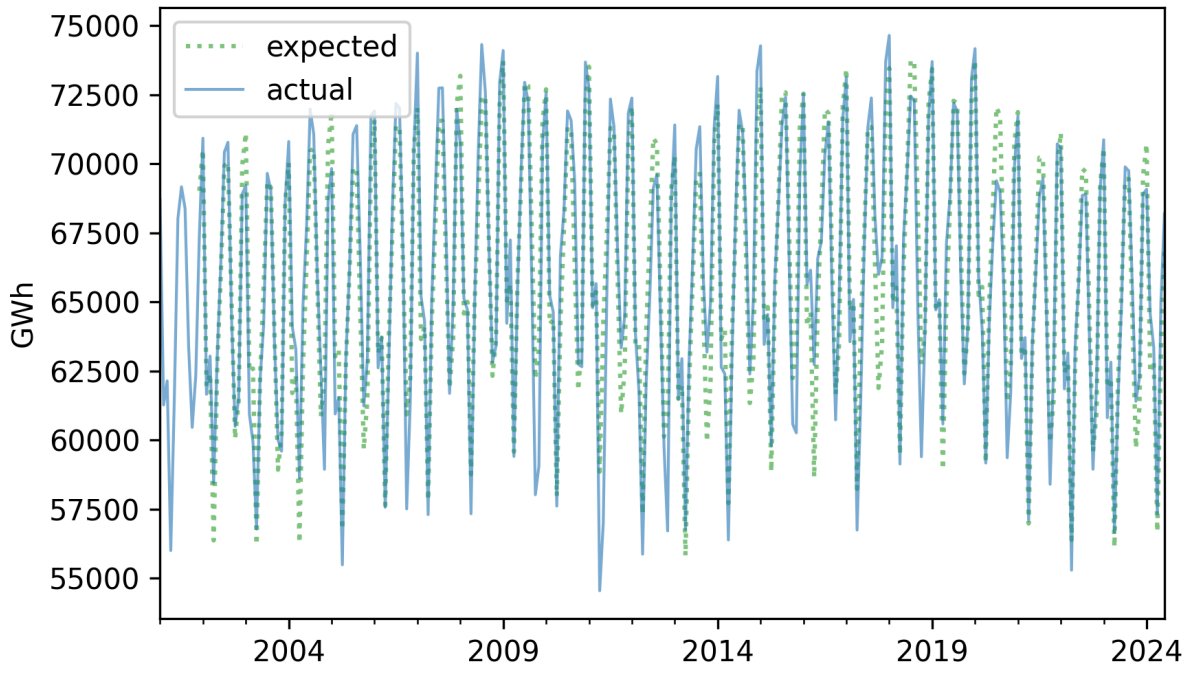
Each 12-month period is identical to the others.

The sum of the trend and the seasonal component represents the expected value for each month.

```
expected = trend + seasonal
```

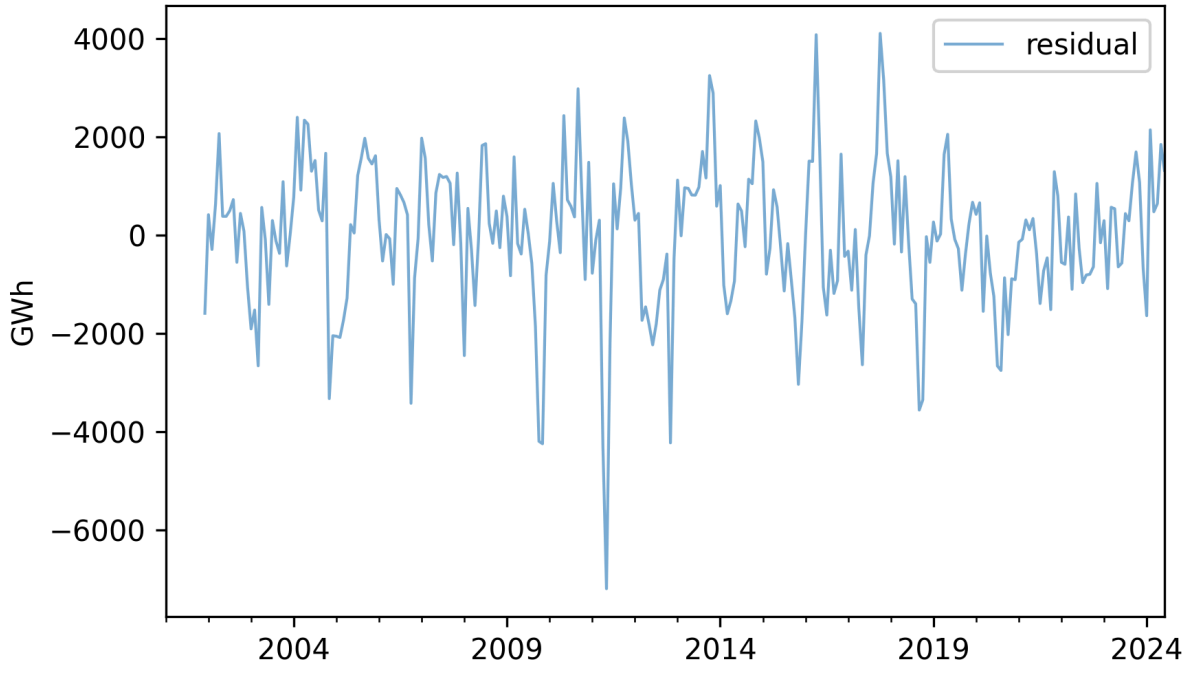
Here's what it looks like compared to the original series.

```
expected.plot(label="expected", **pred_options)
nuclear.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```



If we subtract this sum from the original series, the result is the residual component, which represents the departure from the expected value for each month.

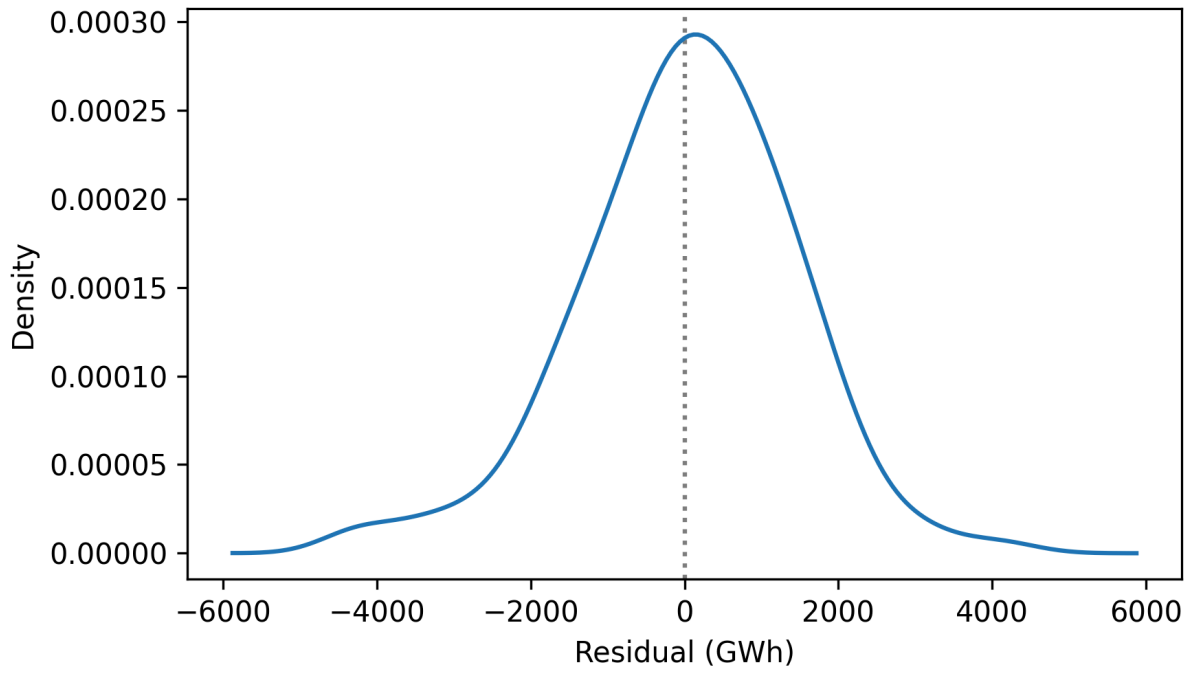
```
resid = nuclear - expected
resid.plot(label="residual", **actual_options)
decorate(ylabel="GWh")
```



We can think of the residual as the sum of everything in the world that affects energy production, but is not explained by the long-term trend or the seasonal component. Among other things, that sum includes weather, equipment that's down for maintenance, and changes in demand due to specific events. Since the residual is the sum of many unpredictable, and sometimes unknowable, factors, we often treat it as a random quantity.

Here's what the distribution of the residuals look like.

```
from thinkstats import plot_kde

plot_kde(resid.dropna())
decorate(xlabel="Residual (GWh)", ylabel="Density")
```



It resembles the bell curve of the normal distribution, which is consistent with the assumption that it is the sum of many random contributions.

To quantify how well this model describes the original series, we can compute the coefficient of determination, which indicates how much smaller the variance of the residuals is, compared to the variance of the original series.

```
rsquared = 1 - resid.var() / nuclear.var()
rsquared
```

```
np.float64(0.9054559977517084)
```

The R^2 value is about 0.92, which means that the long-term trend and seasonal component account for 92% of the variability in the series. This R^2 is substantially higher than the ones we saw in the previous chapter, but that's common with time series data -- especially in a case like this where we've constructed the model to resemble the data.

The process we've just walked through is called **seasonal decomposition**. StatsModels provides a function that does it, called `seasonal_decompose`.

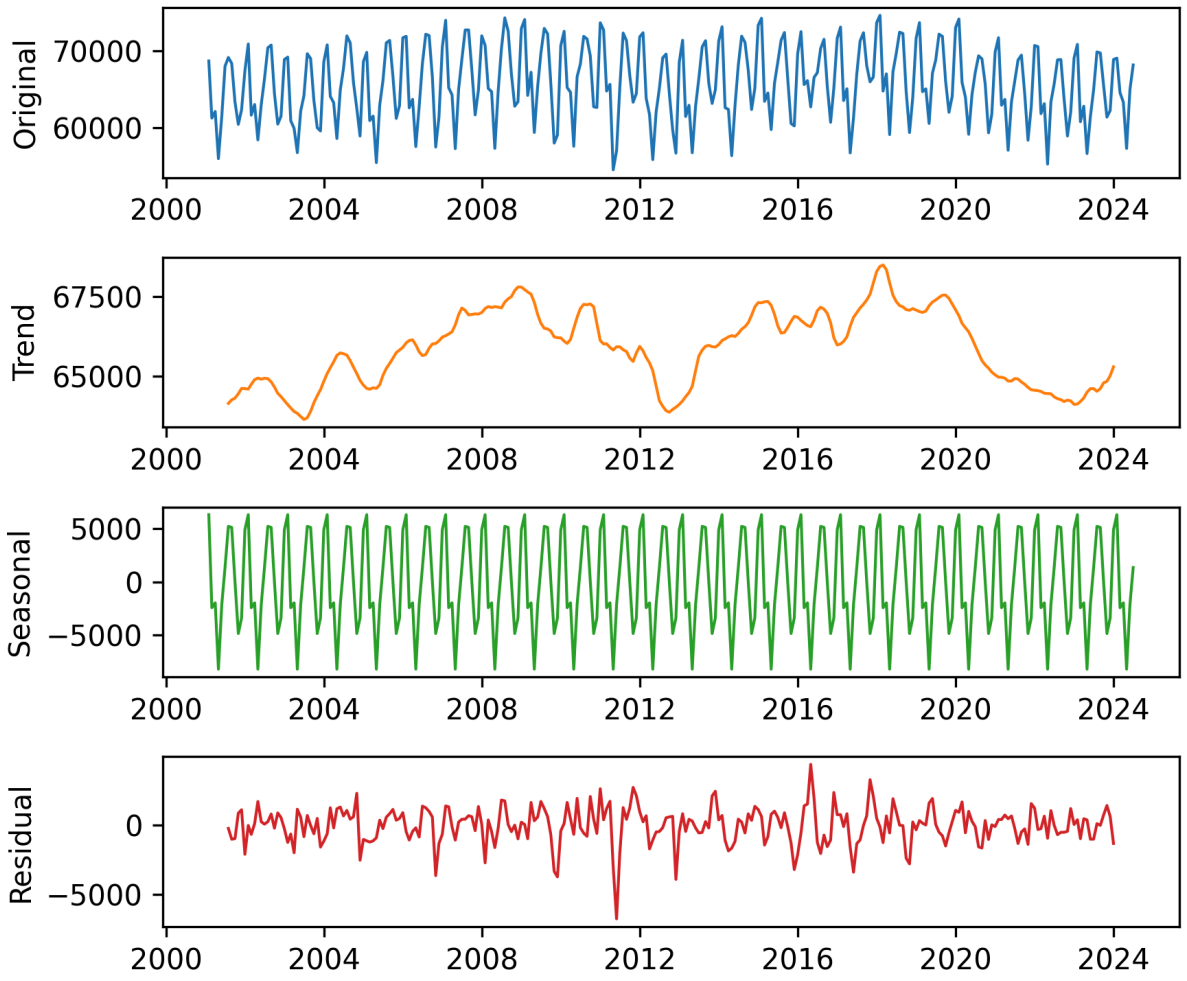
```
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(nuclear, model="additive", period=12)
```

The `model="additive"` argument indicates the additive model, so the series is decomposed into the sum of a trend, seasonal component, and residual. We'll see the multiplicative model soon. The `period=12` argument indicates that the duration of the seasonal component is 12 months.

The result is an object that contains the three components. The notebook for this chapter provides a function that plots them.

```
def plot_decomposition(original, decomposition):
    plt.figure(figsize=(6, 5))

    ax1 = plt.subplot(4, 1, 1)
    plt.plot(original, label="Original", color="C0", lw=1)
    plt.ylabel("Original")

    plt.subplot(4, 1, 2, sharex=ax1)
    plt.plot(decomposition.trend, label="Trend", color="C1", lw=1)
    plt.ylabel("Trend")

    plt.subplot(4, 1, 3, sharex=ax1)
    plt.plot(decomposition.seasonal, label="Seasonal", color="C2", lw=1)
    plt.ylabel("Seasonal")

    plt.subplot(4, 1, 4, sharex=ax1)
    plt.plot(decomposition.resid, label="Residual", color="C3", lw=1)
    plt.ylabel("Residual")

    plt.tight_layout()
```

```
plot_decomposition(nuclear, decomposition)
```



The results are similar to those we computed ourselves, with small differences due to the details of the implementation.

This kind of seasonal decomposition provides insight into the structure of a time series. As we'll see in the next section, it is also useful for making forecasts.

Prediction

We can use the results from seasonal decomposition to predict the future. To demonstrate, we'll use the following function to split the time series into a **training series**, which we'll use to generate predictions, and a **test series**, which we'll use to see whether they are accurate.

```
def split_series(series, n=60):
    training = series.iloc[:-n]
    test = series.iloc[-n:]
    return training, test
```

With `n=60`, the duration of the test series is five years, starting in July 2019.

```
training, test = split_series(nuclear)
test.index[0]
```

```
Timestamp('2019-07-31 00:00:00')
```

Now, suppose it's June 2019 and you are asked to generate a five-year forecast for electricity production from nuclear generators. To answer this question, we'll use the training data to make a model and then use the model to generate predictions. We'll start with a seasonal decomposition of the training data.

```
decomposition = seasonal_decompose(training, model="additive", period=12)
trend = decomposition.trend
```

Now we'll fit a linear model to the trend. The explanatory variable, `months`, is the number of months from the beginning of the series.

```
import statsmodels.formula.api as smf

months = np.arange(len(trend))
data = pd.DataFrame({"trend": trend, "months": months}).dropna()
results = smf.ols("trend ~ months", data=data).fit()
```

Here is a summary of the results.

```
from thinkstats import display_summary

display_summary(results)
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	6.482e+04	131.524	492.869	0.000	6.46e+04	6.51e+04
months	10.9886	1.044	10.530	0.000	8.931	13.046

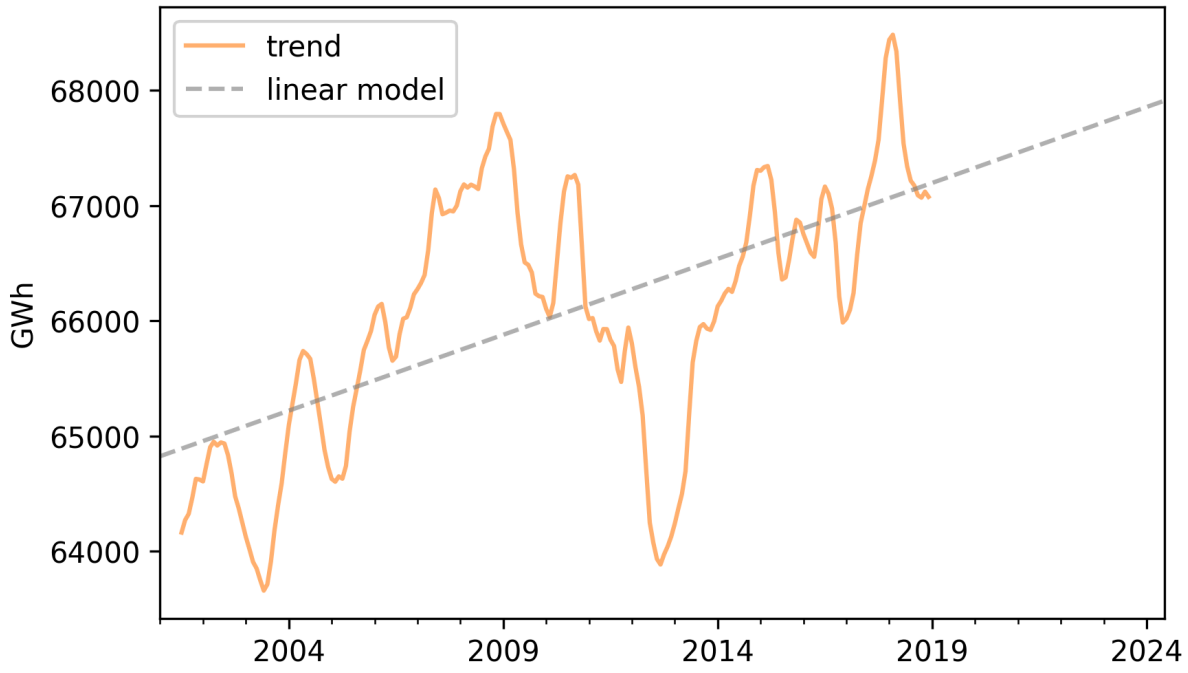
R-squared: 0.3477

The R^2 value is about 0.35, which suggests that the model does not fit the data particularly well. We can get a better sense of that by plotting the fitted line. We'll use the `predict` method to compute expected values for the training and test data.

```
months = np.arange(len(training) + len(test))
df = pd.DataFrame({"months": months})
pred_trend = results.predict(df)
pred_trend.index = nuclear.index
```

Here's the trend component and the linear model.

```
trend.plot(**trend_options)
pred_trend.plot(label="linear model", **model_options)
decorate(ylabel="GWh")
```



There's a lot going on that's not captured by the linear model, but it looks like there is a generally increasing trend.

Next we'll use the seasonal component from the decomposition to compute a `Series` of monthly averages.

```
seasonal = decomposition.seasonal
monthly_averages = seasonal.groupby(seasonal.index.month).mean()
```

We can predict the seasonal component by looking up the dates from the fitted line in `monthly_averages`.

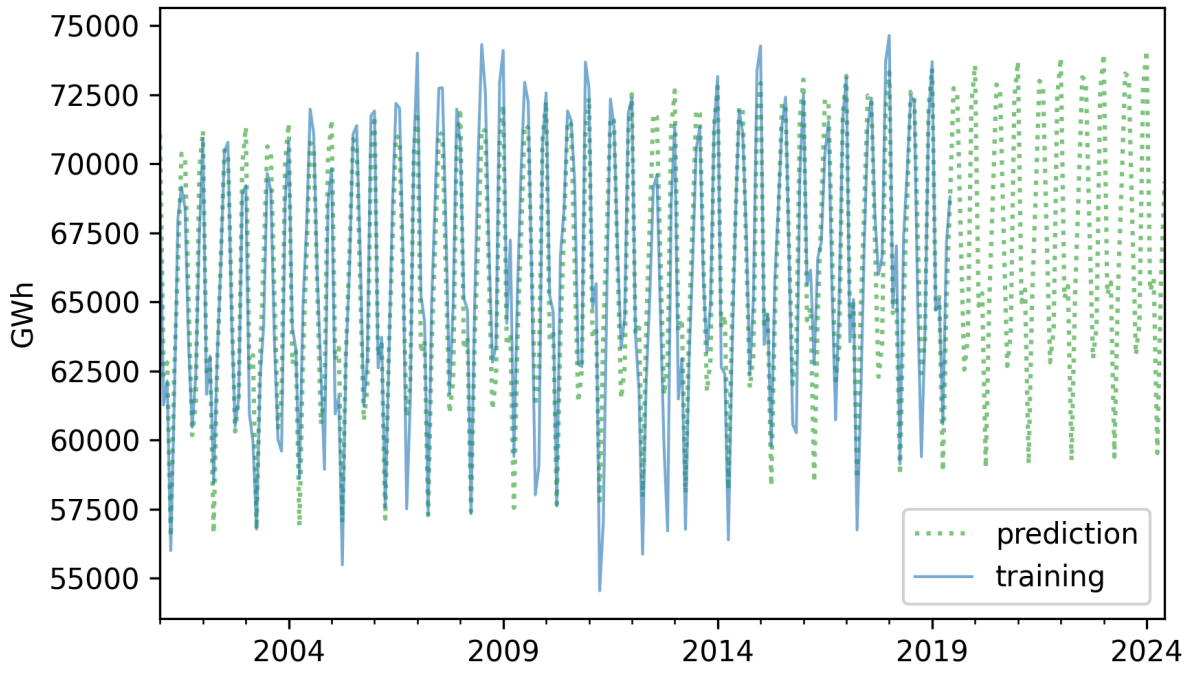
```
pred_seasonal = monthly_averages[pred_trend.index.month]
pred_seasonal.index = pred_trend.index
```

Finally, to generate predictions, we'll add the seasonal component to the trend.

```
pred = pred_trend + pred_seasonal
```

Here's the training data and the predictions.

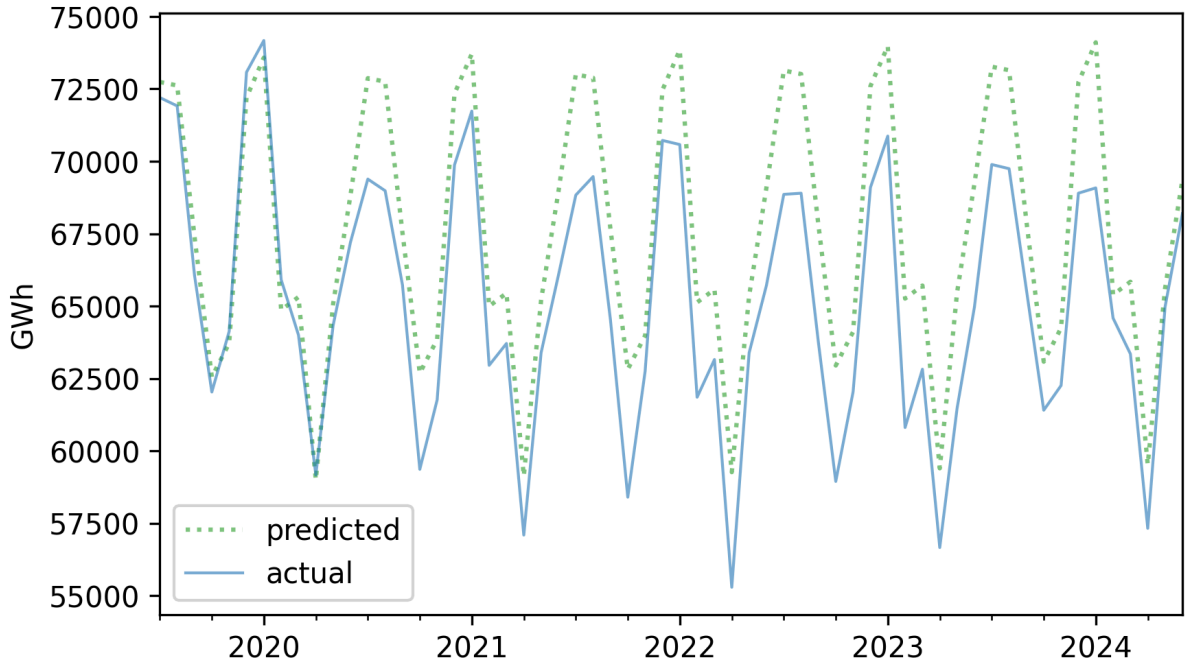
```
pred.plot(label="prediction", **pred_options)
training.plot(label="training", **actual_options)
decorate(ylabel="GWh")
```



The predictions fit the training data reasonably well, and the forecast looks like a reasonable projection, based on the assumption that the long-term trend will continue.

Now, from the vantage point of the future, let's see how accurate this forecast turned out to be. Here are the predicted and actual values for the five-year interval from July 2019.

```
forecast = pred[test.index]
forecast.plot(label="predicted", **pred_options)
test.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```



The first year of the forecast was pretty good, but production from nuclear reactors in 2020 was lower than expected -- possibly due to the COVID-19 pandemic -- and it never returned to the long-term trend.

To quantify the accuracy of the predictions, we'll use the mean absolute percentage error (MAPE), which the following function computes.

```
def MAPE(predicted, actual):
    ape = np.abs(predicted - actual) / actual
    return np.mean(ape) * 100
```

In this example, the predictions are off by 3.81% on average.

```
MAPE(forecast, test)
```

```
np.float64(3.811940747879257)
```

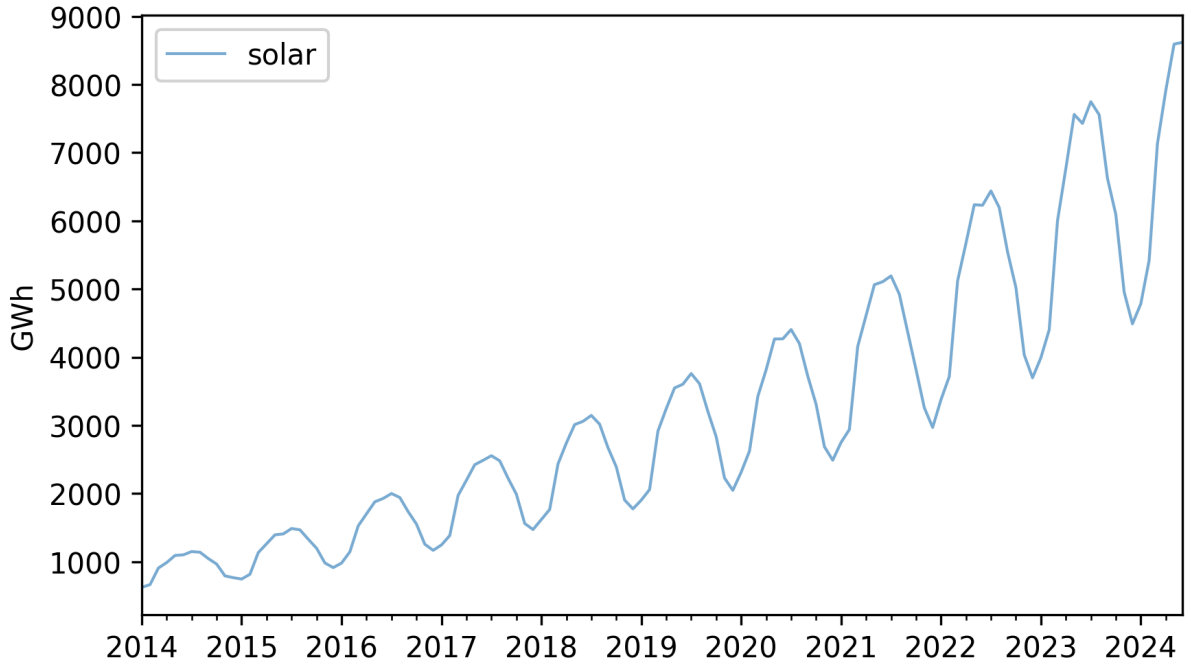
We'll come back to this example later in the chapter and see if we can do better with a different model.

Multiplicative Model

The additive model we used in the previous section assumes that the time series is the *sum* of a long-term trend, a seasonal component, and a residual -- which implies that the magnitude of the seasonal component and the residuals does not vary over time.

As an example that violates this assumption, let's look at small-scale solar electricity production since 2014.

```
solar = elec["United States : small-scale solar photovoltaic"].dropna()
solar.plot(label="solar", **actual_options)
decorate(ylabel="GWh")
```



Over this interval, total production has increased several times over. And it's clear that the magnitude of seasonal variation has increased as well.

If we suppose that the magnitudes of seasonal and random variation are proportional to the magnitude of the trend, that suggests an alternative to the additive model in which the time series is the *product* of the three components.

To try out this multiplicative model, we'll split this series into training and test sets.

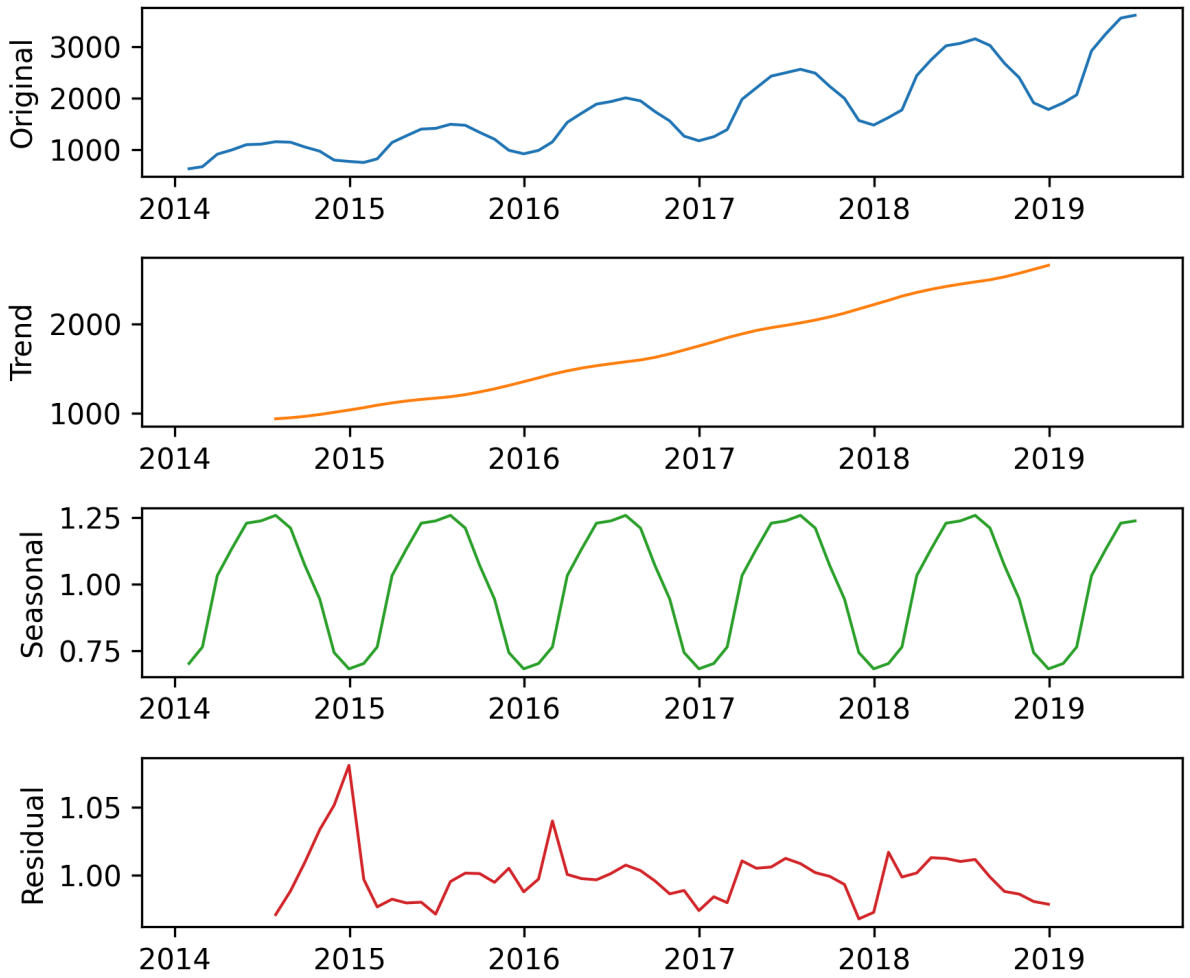
```
training, test = split_series(solar)
```

And call `seasonal_decompose` with the `model="multiplicative"` argument.

```
decomposition = seasonal_decompose(training, model="multiplicative", period=12)
```

Here's what the results look like.

```
plot_decomposition(training, decomposition)
```



Now the seasonal and residual components are multiplicative factors. So, it looks like the seasonal component varies from about 25% below the trend to 25% above. And the residual component is usually less than 5% either way, with the exception of some larger factors in the first period. We can extract the components of the model like this.

```
trend = decomposition.trend
seasonal = decomposition.seasonal
resid = decomposition.resid
```

The R^2 value of this model is very high.

```
rsquared = 1 - resid.var() / training.var()
rsquared
```

```
np.float64(0.9999999992978134)
```

The production of a solar panel is largely a function of the sunlight it's exposed to, so it makes sense that production follows an annual cycle so closely.

To predict the long term trend, we'll use a quadratic model.

```
months = range(len(training))
data = pd.DataFrame({"trend": trend, "months": months}).dropna()
results = smf.ols("trend ~ months + I(months**2)", data=data).fit()
```

In the Patsy formula, the substring `I(months**2)` adds a quadratic term to the model, so we don't have to compute it explicitly. Here are the results.

```
display_summary(results)
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	766.1962	13.494	56.782	0.000	739.106	793.286
months	22.2153	0.938	23.673	0.000	20.331	24.099
I(months ** 2)	0.1762	0.014	12.480	0.000	0.148	0.205

R-squared: 0.9983

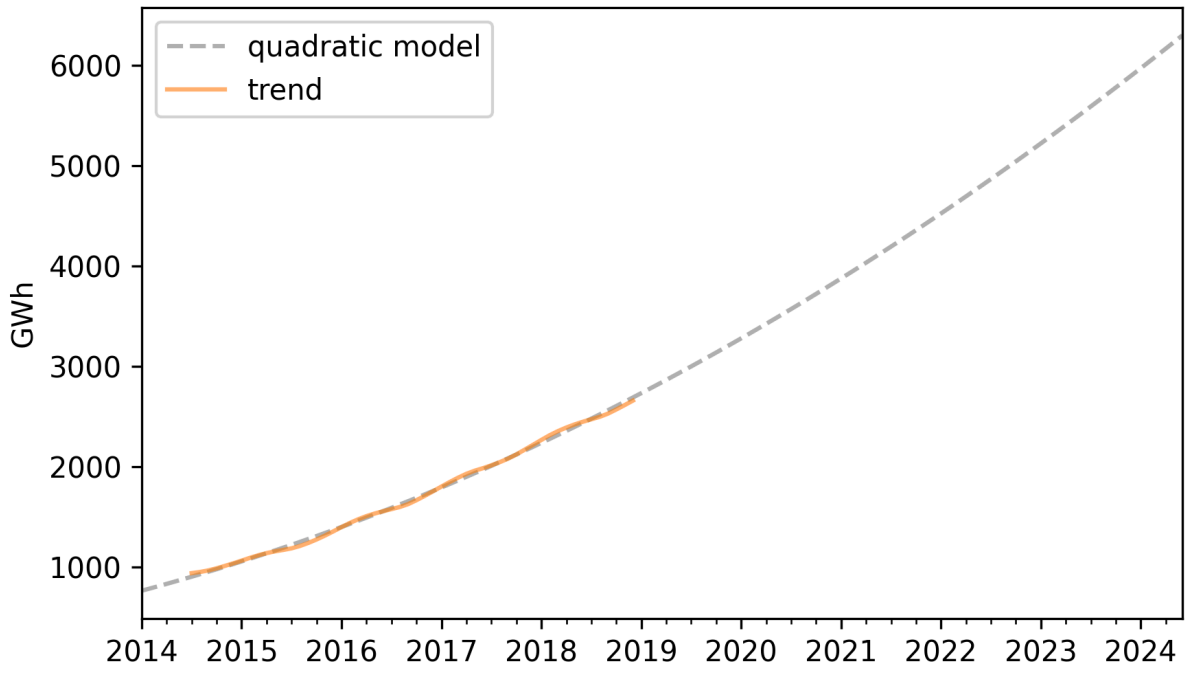
The p-values of the linear and quadratic terms are very small, which suggests that the quadratic model captures more information about the trend than a linear model would -- and the R^2 value is very high.

Now we can use the model to compute the expected value of the trend for the past and future.

```
months = range(len(solar))
df = pd.DataFrame({"months": months})
pred_trend = results.predict(df)
pred_trend.index = solar.index
```

Here's what it looks like.

```
pred_trend.plot(label="quadratic model", **model_options)
trend.plot(**trend_options)
decorate(ylabel="GWh")
```



The quadratic model fits the past trend well. Now we can use the seasonal component to predict future seasonal variation.

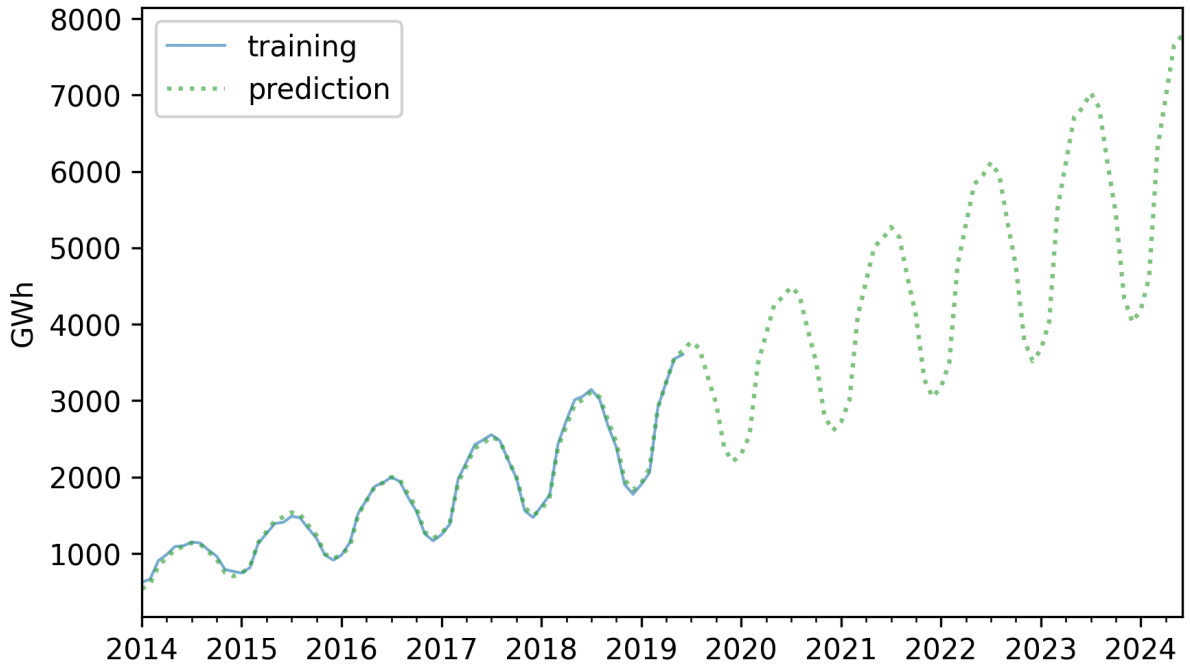
```
monthly_averages = seasonal.groupby(seasonal.index.month).mean()
pred_seasonal = monthly_averages[pred_trend.index.month]
pred_seasonal.index = pred_trend.index
```

Finally, to compute **retrodictions** for past values and predictions for the future, we multiply the trend and the seasonal component.

```
pred = pred_trend * pred_seasonal
```

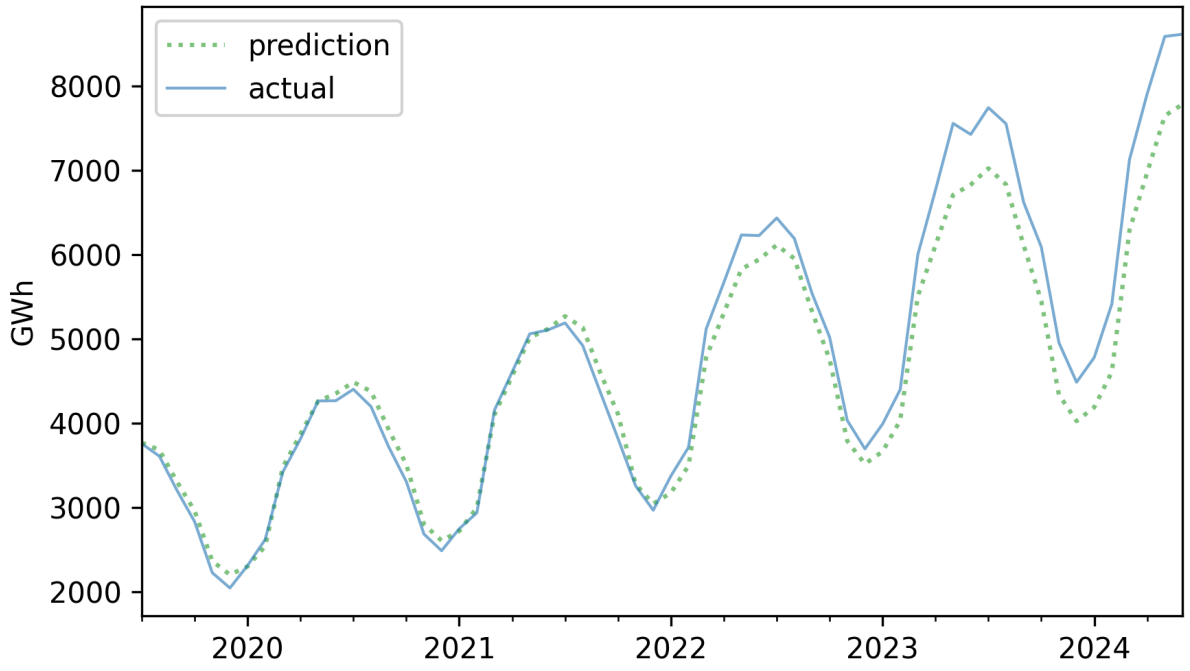
Here is the result along with the training data.

```
training.plot(label="training", **actual_options)
pred.plot(label="prediction", **pred_options)
decorate(ylabel="GWh")
```



The retrodictions fit the training data well and the predictions seem plausible -- now let's see if they turned out to be accurate. Here are the predictions along with the test data.

```
future = pred[test.index]
future.plot(label="prediction", **pred_options)
test.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```



For the first three years, the predictions are very good. After that, it looks like actual growth exceeded expectations.

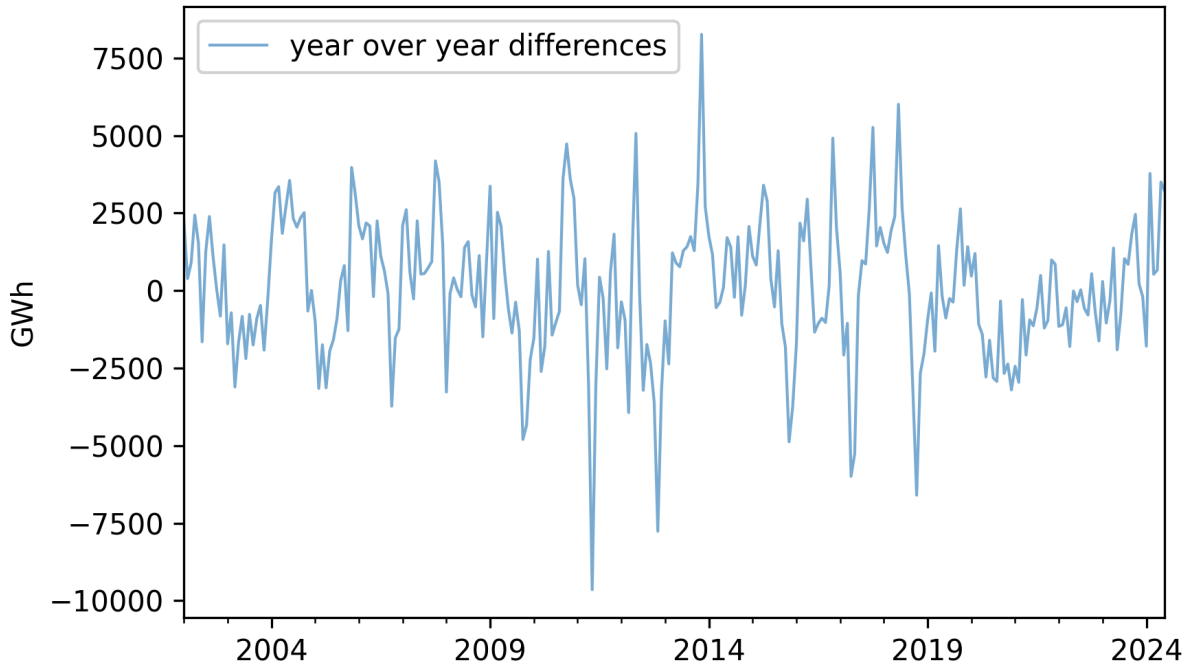
In this example, seasonal decomposition worked well for modeling and predicting solar production, but in the previous example, it was not very effective for nuclear production. In the next section, we'll try a different approach, autoregression.

Autoregression

The first idea of autoregression is that the future will be like the past. For example, in the time series we've looked at so far, there is a clear annual cycle. So if you are asked to make a prediction for next June, a good starting place would be last June.

To see how well that might work, let's go back to `nuclear`, which contains monthly electricity production from nuclear generators, and compute differences between the same month in successive years, which are called "year-over-year" differences.

```
diff = (nuclear - nuclear.shift(12)).dropna()
diff.plot(label="year over year differences", **actual_options)
decorate(ylabel="GWh")
```



The magnitudes of these differences are substantially smaller than the magnitudes of the original series, which suggests the second idea of autoregression, which is that it might be easier to predict these differences, rather than the original values.

Toward that end, let's see if there are correlations between successive elements in the series of differences. If so, we could use those correlations to predict future values based on previous values.

I'll start by making a `DataFrame`, putting the differences in the first column and putting the same differences -- shifted by 1, 2, and 3 months -- into successive columns. These columns are named `lag1`, `lag2`, and `lag3`, because the series they contain have been **lagged** or delayed.

```
df_ar = pd.DataFrame({"diff": diff})
for lag in [1, 2, 3]:
    df_ar[f"lag{lag}"] = diff.shift(lag)

df_ar = df_ar.dropna()
```

Here are the correlations between these columns.

```
df_ar.corr()[["diff"]]
```

	diff
diff	1.000000
lag1	0.562212
lag2	0.292454
lag3	0.222228

These correlations are called lagged correlations or **autocorrelations** -- the prefix "auto" indicates that we're taking the correlation of the series with itself. As a special case, the correlation between `diff` and `lag1` is called **serial correlation** because it is the correlation between successive elements in the series.

These correlation are strong enough to suggest that they should help with prediction, so let's put them into a multiple regression. The following function uses the columns from the `DataFrame` to make a Patsy formula with the first column as the response variable and the other columns as explanatory variables.

```
def make_formula(df):
    """Make a Patsy formula from column names."""
    y = df.columns[0]
    xs = " + ".join(df.columns[1:])
    return f"{y} ~ {xs}"
```

Here are the results of a linear model that predicts the next value in a sequence based on the previous three values.

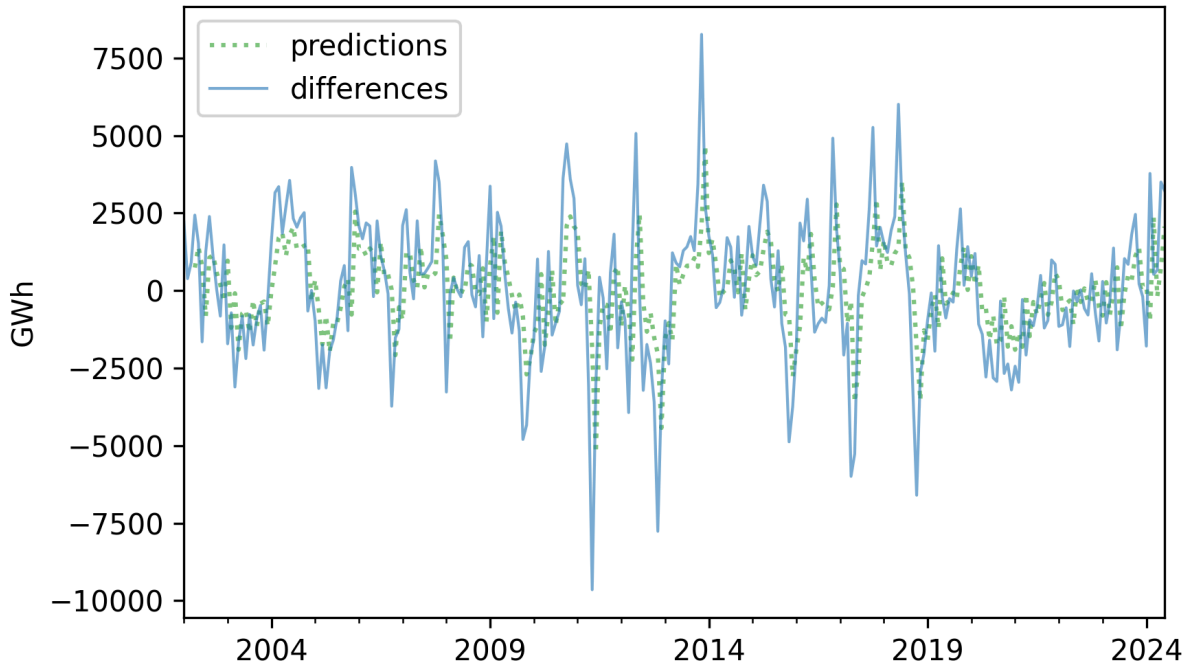
```
formula = make_formula(df_ar)
results_ar = smf.ols(formula=formula, data=df_ar).fit()
display_summary(results_ar)
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	24.2674	114.674	0.212	0.833	-201.528	250.063
lag1	0.5847	0.061	9.528	0.000	0.464	0.706
lag2	-0.0908	0.071	-1.277	0.203	-0.231	0.049
lag3	0.1026	0.062	1.666	0.097	-0.019	0.224

R-squared: 0.3239

Now we can use the `predict` method to generate predictions for the past values in the series. Here's what these retrodictions look like compared to the data.

```
pred_ar = results_ar.predict(df_ar)
pred_ar.plot(label="predictions", **pred_options)
diff.plot(label="differences", **actual_options)
decorate(ylabel="GWh")
```



The predictions are good in some places, but the R^2 value is only about 0.319, so there is room for improvement.

```
resid_ar = (diff - pred_ar).dropna()
R2 = 1 - resid_ar.var() / diff.var()
R2
```

```
np.float64(0.3190252265690783)
```

One way to improve the predictions is to compute the residuals from this model and use another model to predict the residuals -- which is the third idea of autoregression.

Moving Average

Suppose it's June 2019, and you are asked to make a prediction for June 2020. Your first guess might be that this year's value will be repeated next year.

Now suppose it's May 2020, and you are asked to revise your prediction for June 2020. You could use the results from the last three months, and the autocorrelation model from the previous section, to predict the year-over-year difference.

Finally, suppose you check the predictions for the last few months, and see that they have been consistently too low. That suggests that the prediction for next month might also be too low, so you could revise it upward. The underlying assumption is that recent prediction errors predict future prediction errors.

To see whether they do, we can make a `DataFrame` with the residuals from the autoregression model in the first column, and lagged versions of the residuals in the other columns. For this example, I'll use lags of 1 and 6 months.

```
df_ma = pd.DataFrame({"resid": resid_ar})

for lag in [1, 6]:
    df_ma[f"lag{lag}"] = resid_ar.shift(lag)

df_ma = df_ma.dropna()
```

We can use `ols` to make an autoregression model for the residuals. This part of the model is called a "moving average" because it reduces variability in the predictions in a way that's analogous to the effect of a moving average. I don't find that term particularly helpful, but it is conventional.

Anyway, here's a summary of the autoregression model for the residuals.

```
formula = make_formula(df_ma)
results_ma = smf.ols(formula=formula, data=df_ma).fit()
display_summary(results_ma)
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	-14.0016	114.697	-0.122	0.903	-239.863	211.860
lag1	0.0014	0.062	0.023	0.982	-0.120	0.123
lag6	-0.1592	0.063	-2.547	0.011	-0.282	-0.036

R-squared: 0.0247

The R^2 is quite small, so it looks like this part of the model won't help very much. But the p-value for the 6-month lag is small, which suggests that it contributes more information than we'd expect by chance.

Now we can use the model to generate retrodictions for the residuals.

```
pred_ma = results_ma.predict(df_ma)
```

Then, to generate retrodictions for the year-over-year differences, we add the adjustment from the second model to the retrodictions from the first.

```
pred_diff = pred_ar + pred_ma
```

The R^2 value for the sum of the two models is about 0.332, which is just a little better than the result without the moving average adjustment (0.319).

```
resid_ma = (diff - pred_diff).dropna()
R2 = 1 - resid_ma.var() / diff.var()
R2
```

```
np.float64(0.3315101001391231)
```

Next we'll use these year-over-year differences to generate retrodictions for the original values.

Retrodiction with Autoregression

To generate retrodictions, we'll start by putting the year-over-year differences in a `Series` that's aligned with the index of the original.

```
pred_diff = pd.Series(pred_diff, index=nuclear.index)
```

Using `isna` to check for `NaN` values, we find that the first 21 elements of the new `Series` are missing.

```
n_missing = pred_diff.isna().sum()
n_missing
```

```
np.int64(21)
```

That's because we shifted the `Series` by 12 months to compute year-over-year differences, then we shifted the differences 3 months for the first autoregression model, and we shifted the residuals of the first model by 6 months for the second model. Each time we shift a `Series` like this, we lose a few values at the beginning, and the sum of these shifts is 21.

So before we can generate retrodictions, we have to prime the pump by copying the first 21 elements from the original into a new `Series`.

```
pred_series = pd.Series(index=nuclear.index, dtype=float)
pred_series.iloc[:n_missing] = nuclear.iloc[:n_missing]
```

Now we can run the following loop, which fills in the elements from index 21 (which is the 22nd element) to the end. Each element is the sum of the value from the previous year and the predicted year-over-year difference.

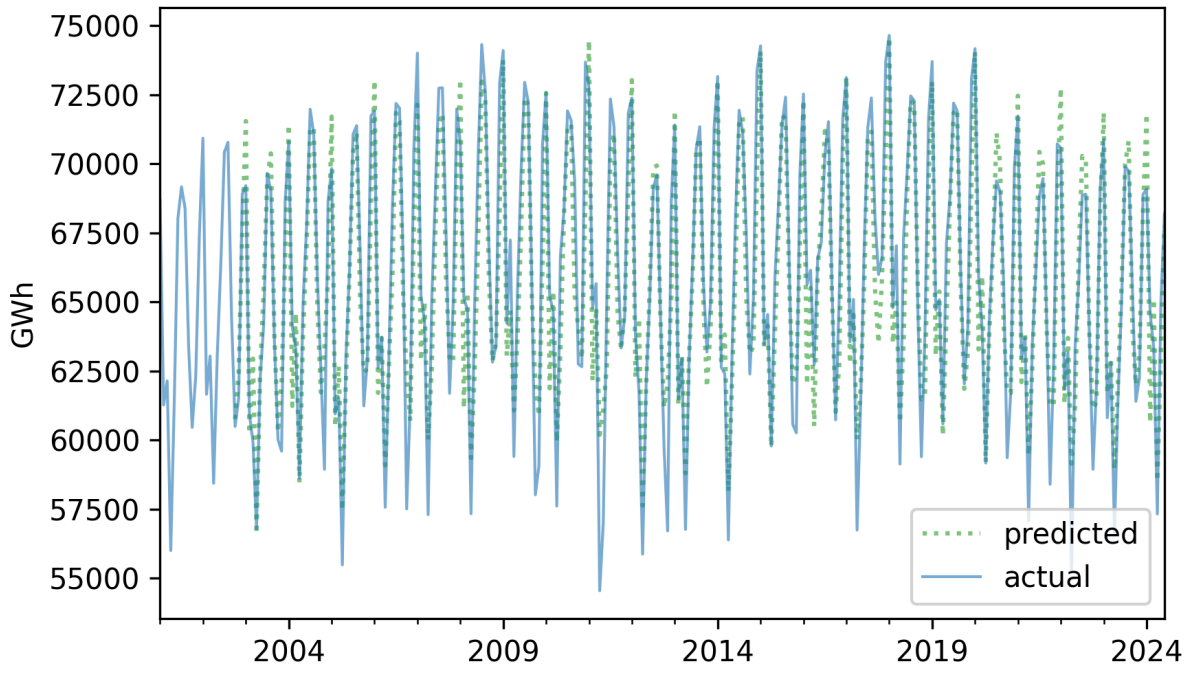
```
for i in range(n_missing, len(pred_series)):
    pred_series.iloc[i] = pred_series.iloc[i - 12] + pred_diff.iloc[i]
```

Now we'll replace the elements we copied with `NaN` so we don't get credit for "predicting" the first 21 values perfectly.

```
pred_series[:n_missing] = np.nan
```

Here's what the retrodictions look like compared to the original.

```
pred_series.plot(label="predicted", **pred_options)
nuclear.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```



They look pretty good, and the R^2 value is about 0.86.

```
resid = (nuclear - pred_series).dropna()
R2 = 1 - resid.var() / nuclear.var()
R2
```

```
np.float64(0.8586566911201012)
```

The model we used to compute these retrodictions is called SARIMA, which is one of a family of models called ARIMA. Each part of these acronyms refers to an element of the model.

- **S** stands for seasonal, because the first step was to compute differences between values separated by one seasonal period.
- **AR** stands for autoregression, which we used to model lagged correlations in the differences.
- **I** stands for integrated, because the iterative process we used to compute `pred_series` is analogous to integration in calculus.
- **MA** stands for moving average, which is the conventional name for the second autoregression model we ran with the residuals from the first.

ARIMA models are powerful and versatile tools for modeling time series data.

ARIMA

StatsModel provides a library called `tsa`, which stands for "time series analysis" -- it includes a function called `ARIMA` that fits ARIMA models and generates forecasts.

To fit the SARIMA model we developed in the previous sections, we'll call this function with two tuples as arguments: `order` and `seasonal_order`. Here are the values in `order` that correspond to the model we used in the previous sections.

```
order = ([1, 2, 3], 0, [1, 6])
```

The values in `order` indicate:

- Which lags should be included in the AR model -- in this example it's the first three.
- How many times it should compute differences between successive elements -- in this example it's 0 because we computed a seasonal difference instead, and we'll get to that in a minute.
- Which lags should be included in the MA model -- in this example it's the first and sixth.

Now here are the values in `seasonal_order`.

```
seasonal_order = (0, 1, 0, 12)
```

The first and third elements are 0, which means that this model does not include seasonal AR or seasonal MA. The second element is 1, which means it computes seasonal differences -- and the last element is the seasonal period.

Here's how we use `ARIMA` to make and fit this model.

```
import statsmodels.tsa.api as tsa

model = tsa.ARIMA(nuclear, order=order, seasonal_order=seasonal_order)
results_arima = model.fit()
display_summary(results_arima)
```

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	0.0458	0.379	0.121	0.904	-0.697	0.788
ar.L2	-0.0035	0.116	-0.030	0.976	-0.230	0.223
ar.L3	0.0375	0.049	0.769	0.442	-0.058	0.133
ma.L1	0.2154	0.382	0.564	0.573	-0.533	0.964
ma.L6	-0.0672	0.019	-3.500	0.000	-0.105	-0.030
sigma2	3.473e+06	1.9e-07	1.83e+13	0.000	3.47e+06	3.47e+06

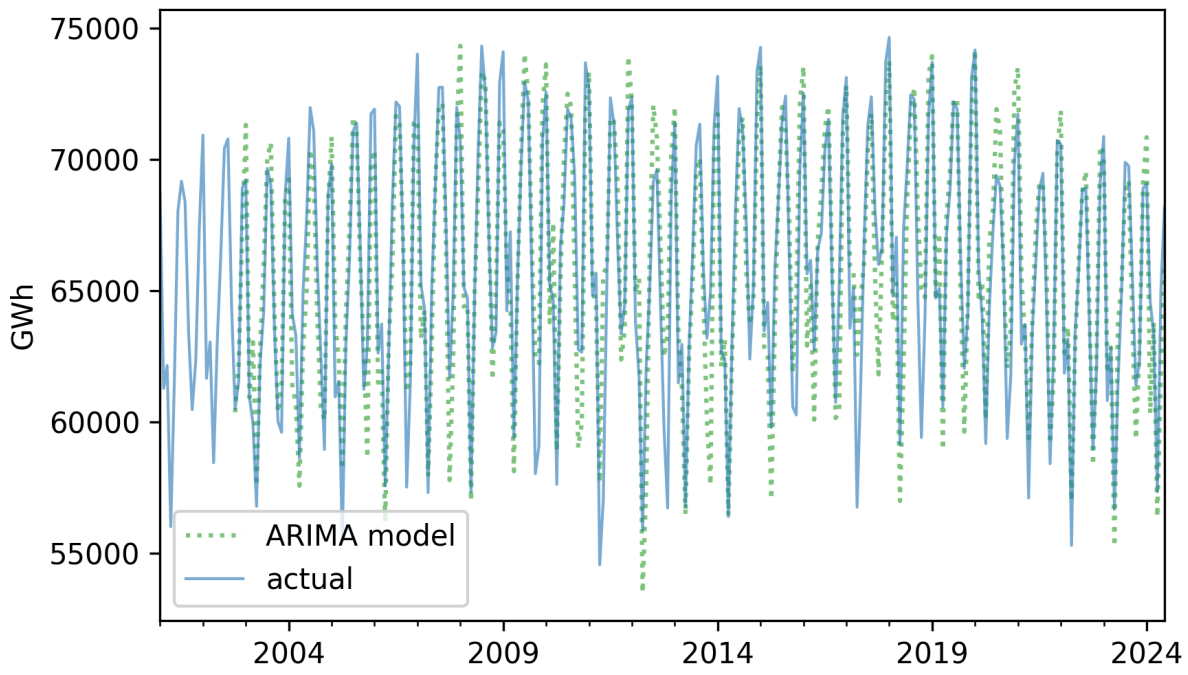
The results include estimated coefficients for the three lags in the AR model, the two lags in the MA model, and `sigma2`, which is the variance of the residuals.

From `results_arima` we can extract `fittedvalues`, which contains the retrodictions. For the same reason there were missing values at the beginning of the retrodictions we computed, there are incorrect values at the beginning of `fittedvalues`, which we'll drop.

```
fittedvalues = results_arima.fittedvalues[n_missing:]
```

The fitted values are similar to the ones we computed, but not exactly the same -- probably because `ARIMA` handles the initial conditions differently.

```
fittedvalues.plot(label="ARIMA model", **pred_options)
nuclear.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```



The R^2 value is also similar but not precisely the same.

```
resid = fittedvalues - nuclear
R2 = 1 - resid.var() / nuclear.var()
R2
```

```
np.float64(0.8262717330736344)
```

The `ARIMA` function makes it easy to experiment with different versions of the model.

As an exercise, try out different values in `order` and `seasonal_order` and see if you can find a model with higher R^2.

Prediction with ARIMA

The object returned by `ARIMA` provides a method called `get_forecast` that generates predictions. To demonstrate, we'll split the time series into a training and test set, and fit the same model to the training set.

```
training, test = split_series(nuclear)
model = tsa.ARIMA(training, order=order, seasonal_order=seasonal_order)
results_training = model.fit()
```

We can use the result to generate a forecast for the test set.

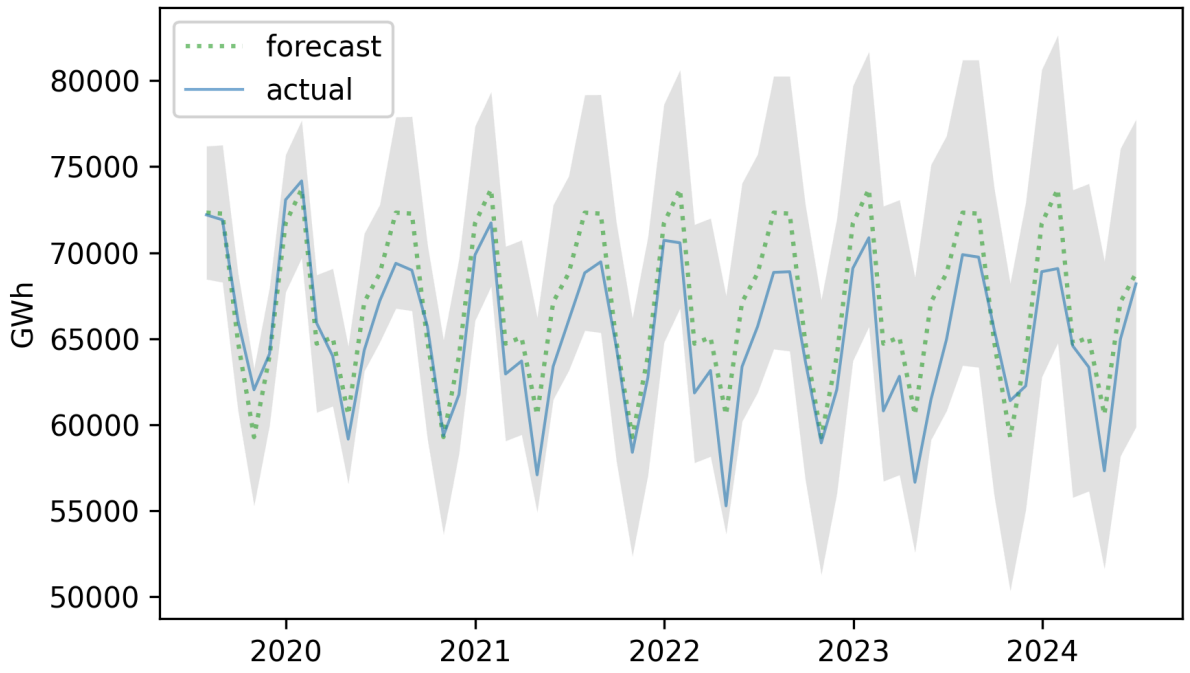
```
forecast = results_training.get_forecast(steps=len(test))
```

The result is an object that contains an attribute called `forecast_mean` and a function that returns a confidence interval.

```
forecast_mean = forecast.predicted_mean
forecast_ci = forecast.conf_int()
forecast_ci.columns = ["lower", "upper"]
```

We can plot the results like this and compare them to the actual time series.

```
plt.fill_between(
    forecast_ci.index,
    forecast_ci.lower,
    forecast_ci.upper,
    lw=0,
    color="gray",
    alpha=0.2,
)
plt.plot(forecast_mean.index, forecast_mean, label="forecast", **pred_options)
plt.plot(test.index, test, label="actual", **actual_options)
decorate(ylabel="GWh")
```



The actual values fall almost entirely within the confidence interval of the predictions. Here's the MAPE of the predictions.

```
MAPE(forecast_mean, test)
```

```
np.float64(3.3817549248342633)
```

The predictions are off by 3.38% on average, somewhat better than the results we got from seasonal decomposition (3.81%).

ARIMA is more versatile than seasonal decomposition, and can often make better predictions. In this time series, the autocorrelations are not especially strong, so the advantage of ARIMA is modest.

Glossary

- **time series**: A dataset where each value is associated with a specific time, often representing measurements taken at regular intervals.
- **seasonal decomposition:** A method of splitting a time series into a long-term trend, a repeating seasonal component, and a residual component.
- **training series:** Part of a time series used to fit a model.
- **test series:** Part of a time series used to check the accuracy of predictions generated by a model.
- **retrodiction:** A prediction for a value observed in the past, often used to test or validate a model.
- **window**: A sequence of consecutive values in a time series, used to compute a moving average.
- **moving average**: A time series computed by averaging values in overlapping windows to smooth fluctuations.
- **serial correlation**: The correlation between successive elements of a time series.
- **autocorrelation**: A correlation between a time series and a shifted or lagged version of itself.
- **lag**: The size of the shift in a serial correlation or autocorrelation.

Exercises

Exercise 12.1

As an example of seasonal decomposition, let's model monthly average surface temperatures in the United States. We'll use a dataset from Our World in Data that includes "temperature [in Celsius] of the air measured 2 meters above the ground, encompassing land, sea, and in-land water surfaces," for most countries in the world from 1950 to 2024. Instructions for downloading the data are in the notebook for this chapter.

```
# The following cell downloads data prepared by Our World in Data,
# which I Downloaded September 18, 2024
# from https://ourworldindata.org/grapher/average-monthly-surface-temperature

# Based on modified data from Copernicus Climate Change Service information (2019)
# with "major processing" by Our World in Data
```

```
filename = "monthly-average-surface-temperatures-by-year.csv"
download("https://github.com/AllenDowney/ThinkStats/raw/v3/data/" + filename)
```

We can read the data like this.

```
temp = pd.read_csv("monthly-average-surface-temperatures-by-year.csv")
```

```
temp.head()
```

(part 1 of 3)

	Entity	Code	Year	2024	2023	2022	2021	2020	201
0	Afghanistan	AFG	1	3.300064	-4.335608	-0.322859	-1.001608	-2.560545	0.58
1	Afghanistan	AFG	2	1.024550	4.187041	2.165870	5.688000	2.880046	0.06
2	Afghanistan	AFG	3	5.843506	10.105444	10.483686	9.777976	6.916731	5.75
3	Afghanistan	AFG	4	11.627398	14.277164	17.227650	15.168276	12.686832	13.8
4	Afghanistan	AFG	5	18.957850	19.078170	19.962734	19.885902	18.884047	18.4

(part 2 of 3)

	2018	...	1959	1958	1956	1954	1952	1957	1955
0	1.042471	...	-2.333814	0.576404	-3.351925	-2.276692	-2.812619	-4.239172	-2.191
1	3.622793	...	-1.545529	0.264962	0.455350	-0.304205	0.798226	-2.747945	1.999
2	10.794412	...	5.942937	7.716459	5.090270	4.357703	4.796146	4.434027	7.066
3	14.321226	...	13.752827	14.712909	11.982360	12.155265	13.119270	8.263829	10.41
4	18.100782	...	17.388723	16.352045	20.125462	18.432117	17.614851	15.505956	15.59

(part 3 of 3)

	1953	1951	1950
0	-2.915993	-3.126317	-2.655707
1	1.983414	-2.642800	-3.996040
2	4.590406	3.054388	3.491112
3	11.087193	9.682878	8.332797
4	17.865084	17.095737	17.329062

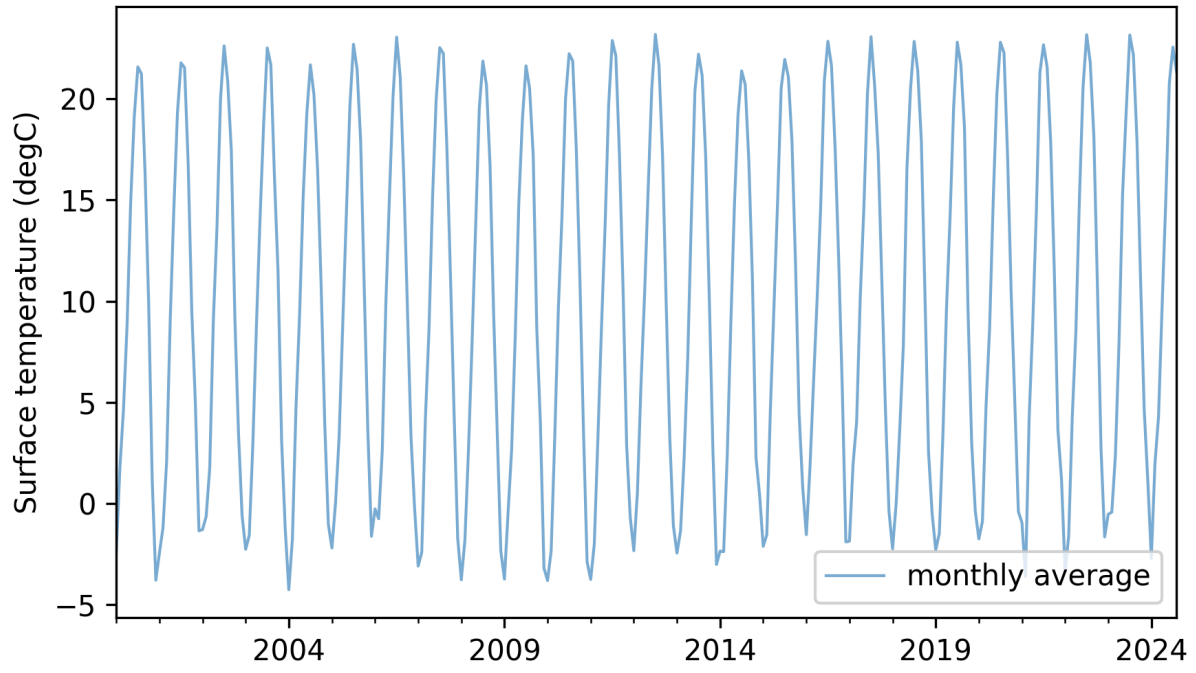
5 rows × 78 columns

The following cell selects data for the United States from 2001 to the end of the series and packs it into a Pandas `Series`.

```
temp_us = temp.query("Code == 'USA'")
columns = [str(year) for year in range(2000, 2025)]
temp_series = temp_us.loc[:, columns].transpose().stack()
temp_series.index = pd.date_range(start="2000-01", periods=len(temp_series), freq="ME")
```

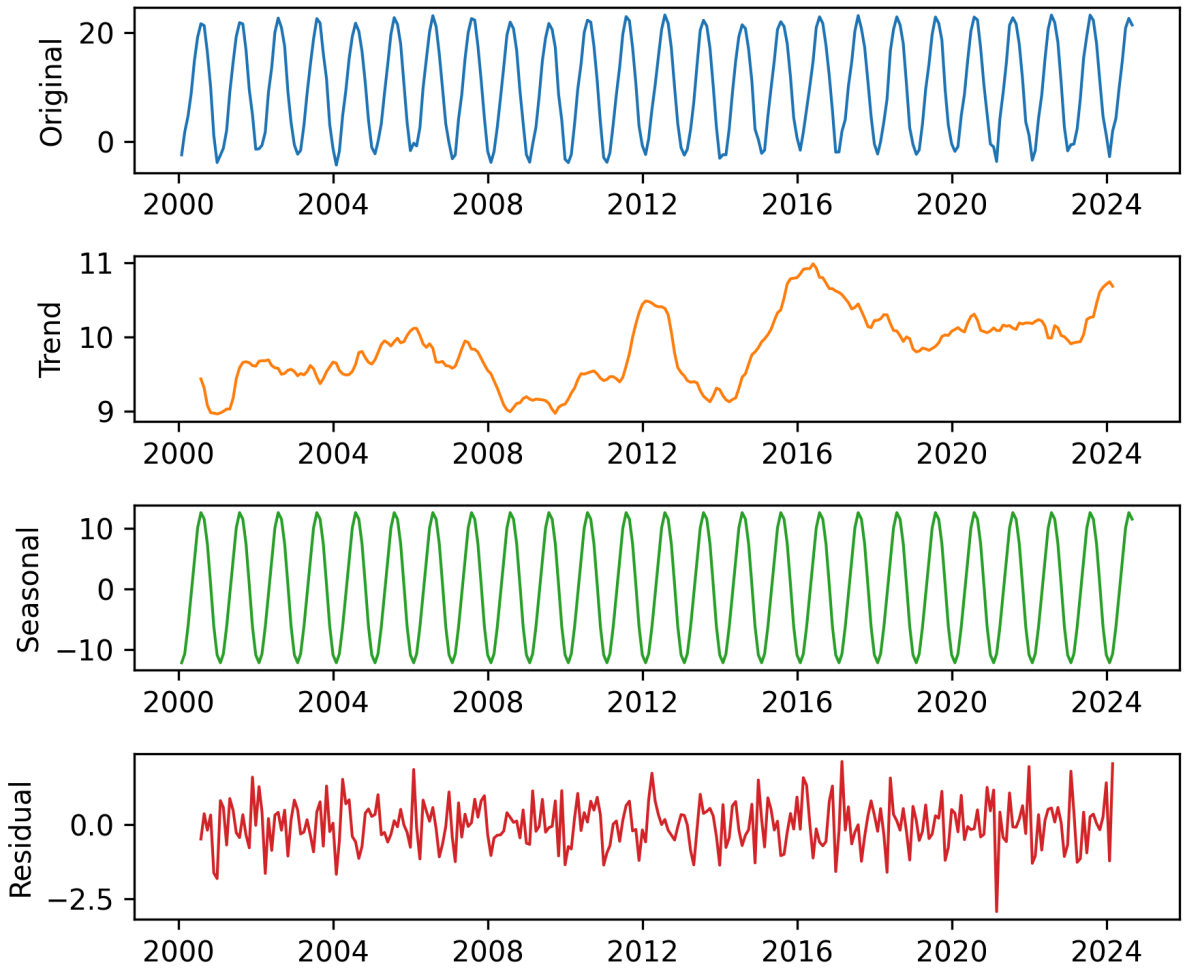
Here's what it looks like.

```
temp_series.plot(label="monthly average", **actual_options)
decorate(ylabel="Surface temperature (degC)")
```



Not surprisingly, there is a strong seasonal pattern. Compute an additive seasonal decomposition with a period of 12 months. Fit a linear model to the trend line. What is the average annual increase in surface temperature during this interval? If you are curious, repeat this analysis with other intervals or data from other countries.

```
# Solution

decomposition = seasonal_decompose(temp_series, model="additive", period=12)
plot_decomposition(temp_series, decomposition)
```




```
# Solution

trend = decomposition.trend
months = np.arange(len(trend))
data = pd.DataFrame({"trend": trend, "months": months}).dropna()
results = smf.ols("trend ~ months", data=data).fit()
display_summary(results)
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	9.3238	0.048	195.796	0.000	9.230	9.418
months	0.0034	0.000	12.036	0.000	0.003	0.004

R-squared: 0.3394

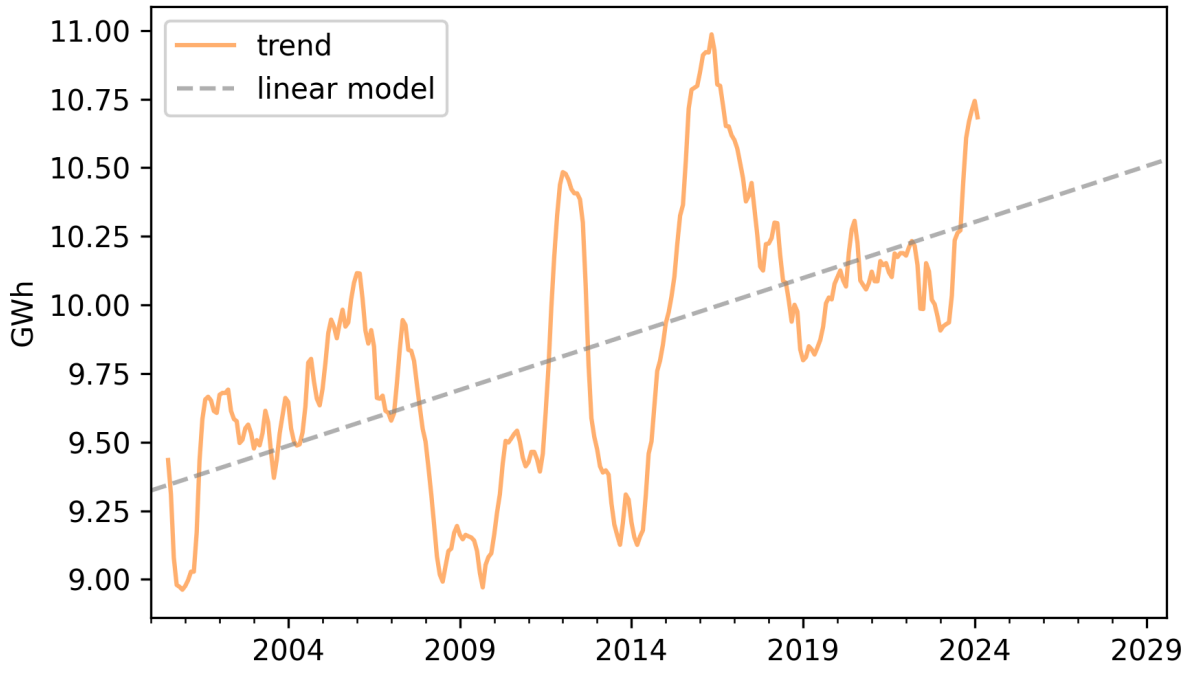
```
# Solution

# Average annual increase in degC

results.params["months"] * 12
```

```
np.float64(0.040756787070318684)
```

```
# Solution

# The question doesn't ask for a forecast, but here's how we can generate one.

months = np.arange(len(trend) + 60)
df = pd.DataFrame({"months": months})
pred_trend = results.predict(df)
```

```
# Solution

# We have to construct an index for the results

t0 = temp_series.index[0]
pred_trend.index = pd.date_range(start=t0, periods=len(pred_trend), freq="ME")
```

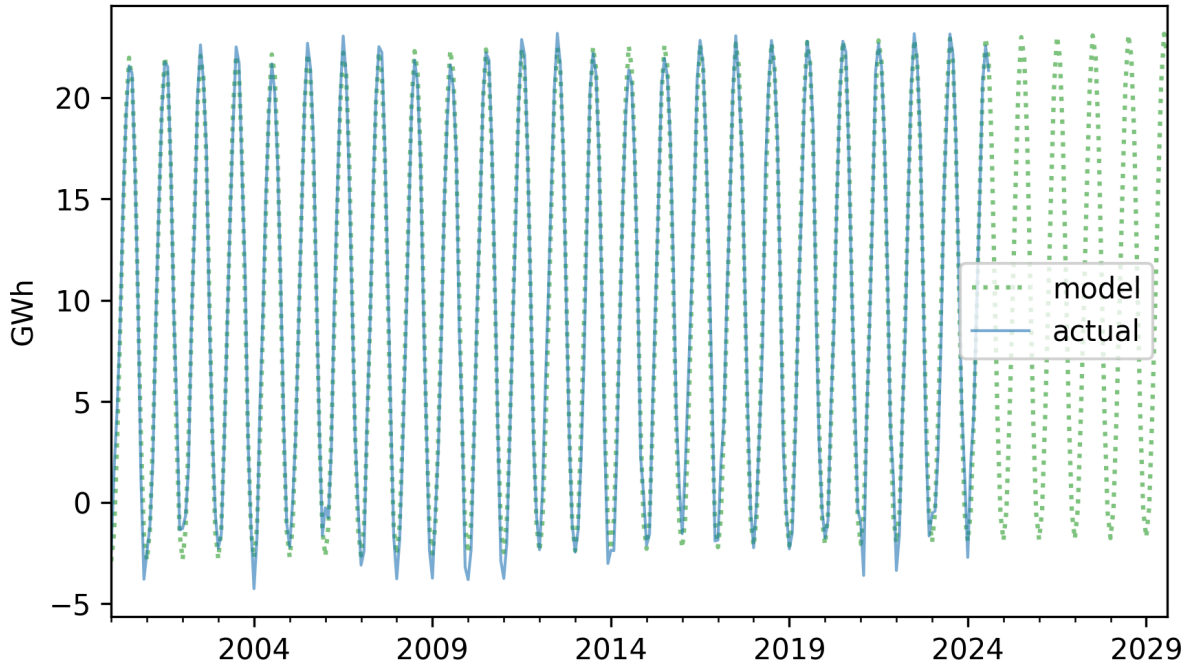
```
# Solution

trend.plot(**trend_options)
pred_trend.plot(label="linear model", **model_options)
decorate(ylabel="GWh")
```




```
# Solution

seasonal = decomposition.seasonal
monthly_averages = seasonal.groupby(seasonal.index.month).mean()
```

```
# Solution

pred_seasonal = monthly_averages[pred_trend.index.month]
pred_seasonal.index = pred_trend.index
```

```
# Solution

pred = pred_trend + pred_seasonal

pred.plot(label="model", **pred_options)
temp_series.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```



COMMENTARY

`seasonal_decompose` with `model="additive"` assumes the observed series can be written as

$$y_t = T_t + S_t + \varepsilon_t$$

where T_t is a slowly-varying trend, S_t is a periodic seasonal component, and ε_t is a residual. Internally, the function estimates T_t using a centred moving average over the specified period (12 months here), which is a low-pass filter that removes oscillations at the seasonal frequency. S_t is then estimated by averaging the detrended values within each calendar month across all years, producing 12 fixed offsets that repeat identically. This is the classical Census X-11 decomposition logic.

The critical hidden assumption is **stationarity of the seasonal amplitude**: the additive model treats the 12 month-offsets as constant across the entire sample. For temperature, this is defensible because the seasonal swing (summer–winter contrast) does not obviously grow with the trend. If you tried this on the solar data you would see the assumption fail badly.

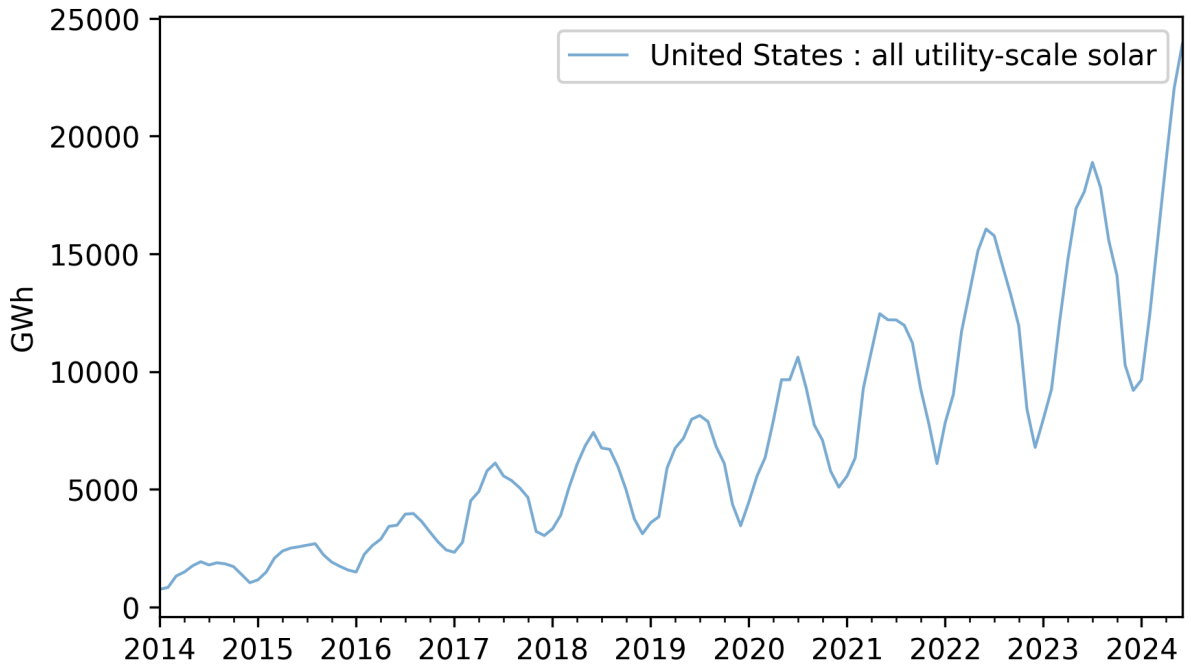
The linear OLS fit on the trend — `smf.ols("trend ~ months", data=data).fit()` — uses ordinary least squares on the moving-average trend rather than on the raw series. This is intentional: the moving average has already removed the seasonal variation, so the regression is not confounded by it. The explanatory variable `months` is a simple integer index, so the fitted coefficient $\hat{\beta}_1$ has units of °C per month. Multiplying by 12 converts to an annual rate. The result (approximately 0.041 °C per year over 2000–2024) is a descriptive estimate, not a causal inference — it captures whatever combination of anthropogenic forcing, natural variability, and local measurement effects operated over this window.

Two subtleties are worth flagging. First, the moving average leaves `NaN` values at both ends of the trend series (six months at each end for a 12-month window), so `.dropna()` is necessary before fitting. Second, the R^2 from regressing on the moving-average trend is not the same as the R^2 of the full model against the raw data — the decomposition has already absorbed seasonal variance, so the linear fit is only being judged against residual trend variability.

Exercise 12.2

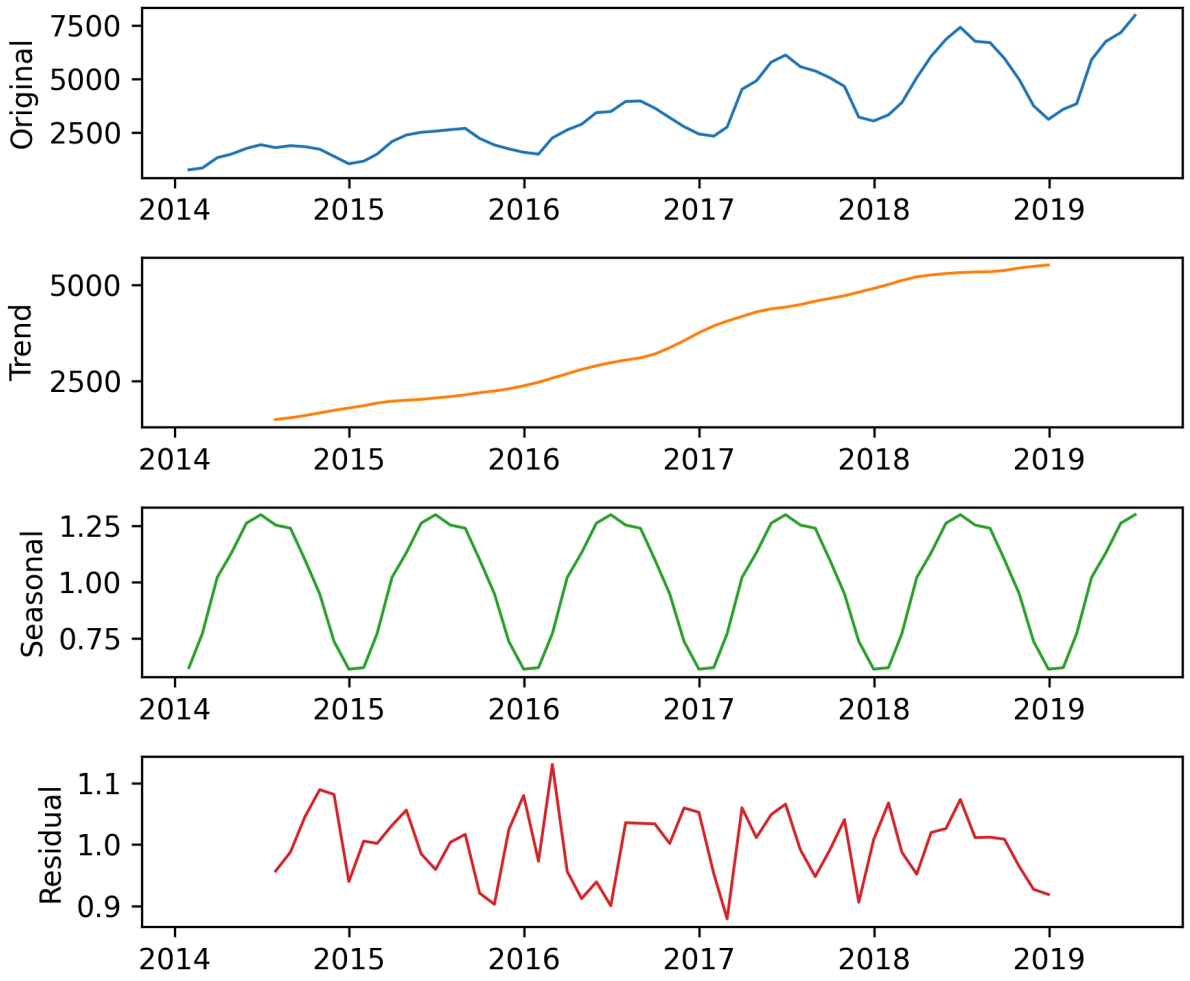
Earlier in this chapter we used a multiplicative seasonal decomposition to model electricity production from small-scale solar power from 2014 to 2019 and forecast production from 2019 to 2024. Now let's do the same with utility-scale solar power. Here's what the time series looks like.

```
util_solar = elec["United States : all utility-scale solar"].dropna()
util_solar = util_solar[util_solar.index.year >= 2014]
util_solar.plot(**actual_options)
decorate(ylabel="GWh")
```



Use `split_series` to split this data into a training and test series. Compute a multiplicative decomposition of the training series with a 12-month period. Fit a linear or quadratic model to the trend and generate a five-year forecast, including a seasonal component. Plot the forecast along with the test series, and compute the mean absolute percentage error (MAPE).

```
# Solution

training, test = split_series(util_solar)
decomposition = seasonal_decompose(training, model="multiplicative", period=12)
plot_decomposition(training, decomposition)
```




```
# Solution

trend = decomposition.trend
seasonal = decomposition.seasonal
resid = decomposition.resid
```

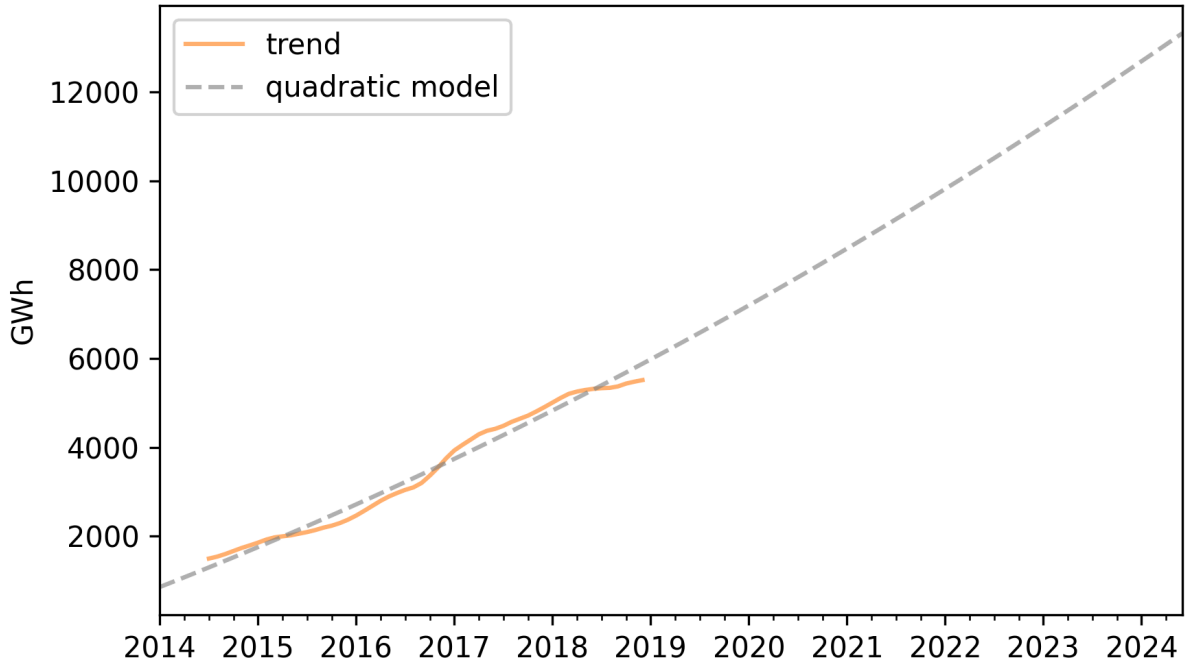
```
# Solution

months = np.arange(len(training))
data = pd.DataFrame({"trend": trend, "months": months}).dropna()
results = smf.ols("trend ~ months + I(months**2)", data=data).fit()
display_summary(results)
```

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	851.0532	115.270	7.383	0.000	619.640	1082.467
months	72.2559	8.016	9.014	0.000	56.162	88.349
I(months ** 2)	0.2205	0.121	1.828	0.073	-0.022	0.463

R-squared: 0.9812

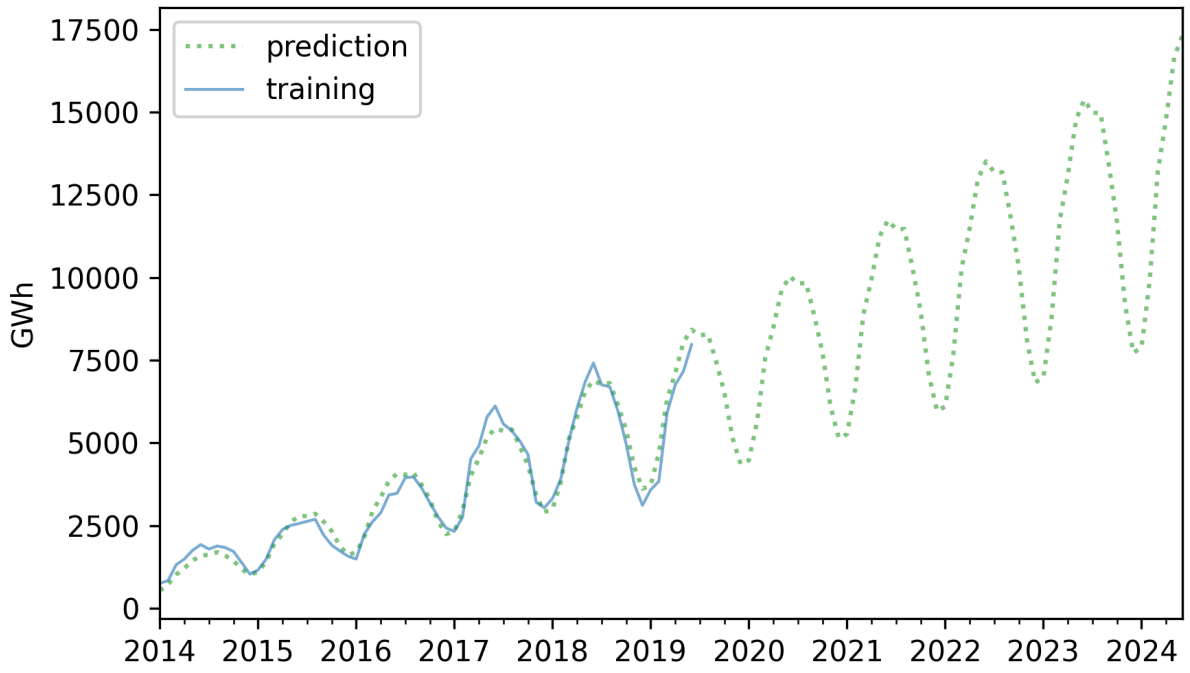
```
# Solution

months = np.arange(len(util_solar))
df = pd.DataFrame({"months": months})
pred_trend = results.predict(df)
pred_trend.index = util_solar.index
```

```
# Solution

trend.plot(**trend_options)
pred_trend.plot(label="quadratic model", **model_options)
decorate(ylabel="GWh")
```

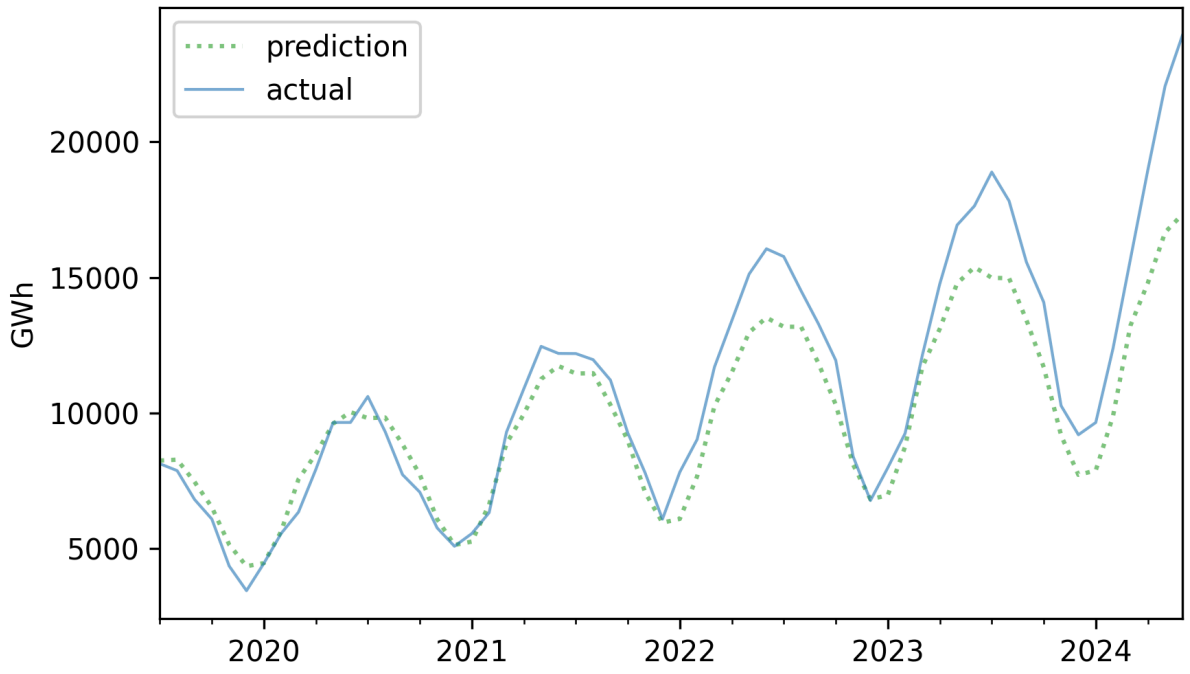



```
# Solution

monthly_averages = seasonal.groupby(seasonal.index.month).mean()
pred_seasonal = monthly_averages[pred_trend.index.month]
pred_seasonal.index = pred_trend.index
pred = pred_trend * pred_seasonal
```

```
# Solution

pred.plot(label="prediction", **pred_options)
training.plot(label="training", **actual_options)
decorate(ylabel="GWh")
```




```
# Solution

# The results for the first few years are pretty good -- after that,
# actual production exceeds the predictions

future = pred[test.index]
future.plot(label="prediction", **pred_options)
test.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```



```
# Solution

MAPE(future, test)
```

```
np.float64(10.676140274760963)
```

COMMENTARY

The choice of `model="multiplicative"` encodes the assumption that the series satisfies

$$y_t = T_t \cdot S_t \cdot \varepsilon_t$$

meaning the seasonal component and the noise are proportional to the level of the trend. Equivalently, $\log y_t = \log T_t + \log S_t + \log \varepsilon_t$, which is an additive model on the log scale. Utility-scale solar grew by roughly an order of magnitude over the training window, so a fixed additive seasonal offset would make no physical sense: a summer surplus of 1 GWh is negligible when total output is 5 GWh but meaningful when it is 50 GWh. The multiplicative model captures this by expressing the seasonal component as a dimensionless ratio (approximately 0.75 in winter, 1.25 in summer) rather than an absolute deviation.

The quadratic trend model, `smf.ols("trend ~ months + I(months**2)", data=data).fit()`, adds a second-order polynomial term. The `I()` wrapper in Patsy is required because `**` in a formula string otherwise denotes a crossing interaction between variables, not exponentiation; `I(months**2)` forces the expression to be evaluated as pure Python, yielding the squared integer. The quadratic fits the accelerating growth phase of solar adoption better than a linear model, which would underestimate recent values. Both the linear and quadratic coefficients are highly significant, confirming that curvature is real in the training window.

Forecasting proceeds by evaluating the fitted polynomial at future integer month indices and multiplying by the average seasonal ratios:

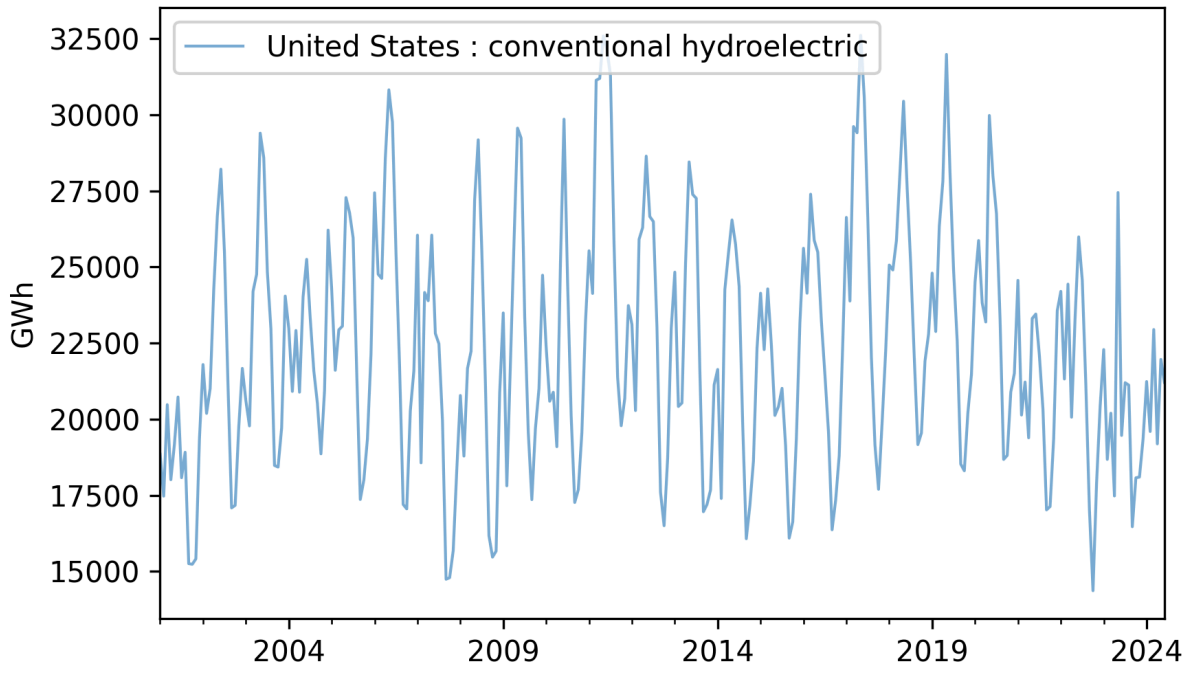
$$\hat{y}t = \hat{T}(t) \cdot \bar{S}_{m(t)}$$

where $m(t)$ is the calendar month of time t and $\bar{S}_m$ is the mean seasonal factor for that month. The MAPE of roughly 10.7% (versus 3.8% for nuclear in the same framework) reflects a fundamental limitation: a polynomial extrapolated beyond its fitting range eventually diverges, and the quadratic underestimates continued near-exponential adoption. The exercise illustrates that model form selection is not just about in-sample fit but about whether the assumed functional form is physically reasonable over the forecast horizon.

Exercise 12.3

Let's see how well an ARIMA model fits production from hydroelectric generators in the United States. Here's what the time series looks like from 2001 to 2024.

```
hydro = elec["United States : conventional hydroelectric"]
hydro.plot(**actual_options)
decorate(ylabel="GWh")
```



Fit a SARIMA model to this data with a seasonal period of 12 months. Experiment with different lags in the autoregression and moving average parts of the model and see if you can find a combination that maximizes the R^2 value of the model. Generate a five-year forecast and plot it along with its confidence interval.

NOTE: Depending on what lags you include in the model, you might find that the first 12 to 24 elements of the fitted values are not reliable. You might want to remove them before plotting or computing R^2.

```
# Solution

model = tsa.ARIMA(hydro, order=([1, 6], 0, [6]), seasonal_order=(0, 1, 0, 12))
results_arima = model.fit()
display_summary(results_arima)
```

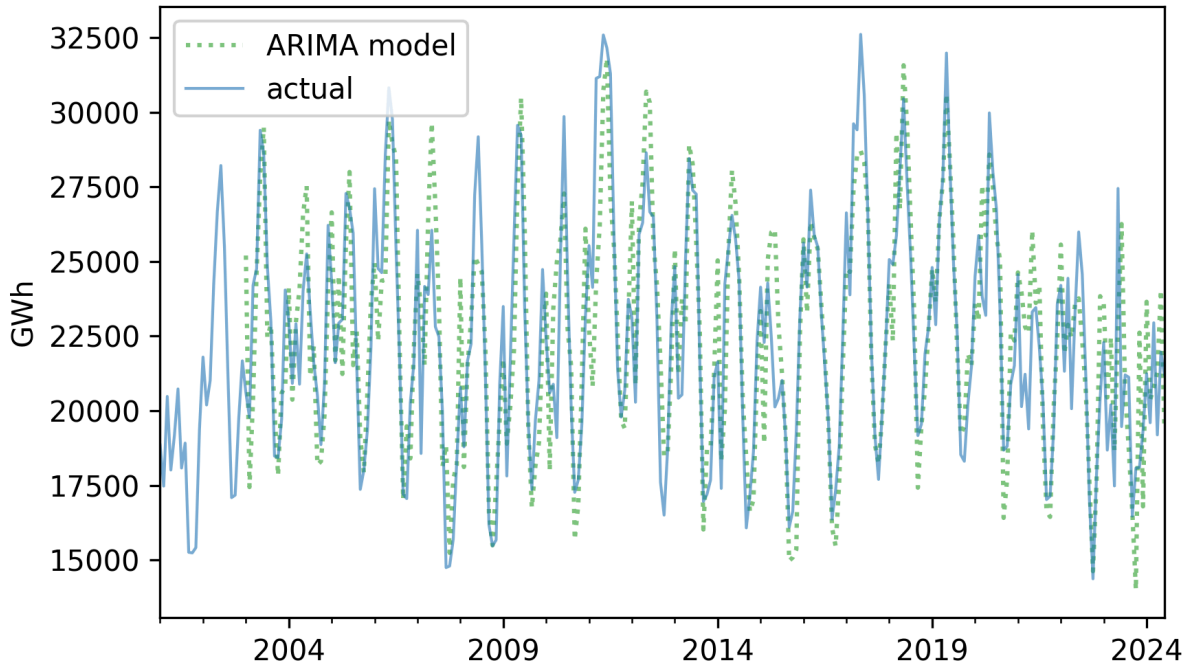
	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	0.5387	0.039	13.680	0.000	0.461	0.616
ar.L6	-0.4041	0.046	-8.722	0.000	-0.495	-0.313
ma.L6	0.9405	0.043	22.009	0.000	0.857	1.024
sigma2	4.437e+06	3.58e+05	12.396	0.000	3.74e+06	5.14e+06

```
# Solution

n_missing = 24
fittedvalues = results_arima.fittedvalues[n_missing:]
```

```
# Solution

resid = hydro - fittedvalues
R2 = 1 - resid.var() / nuclear.var()
R2
```

```
np.float64(0.8057348447529014)
```

```
# Solution

fittedvalues.plot(label="ARIMA model", **pred_options)
hydro.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```

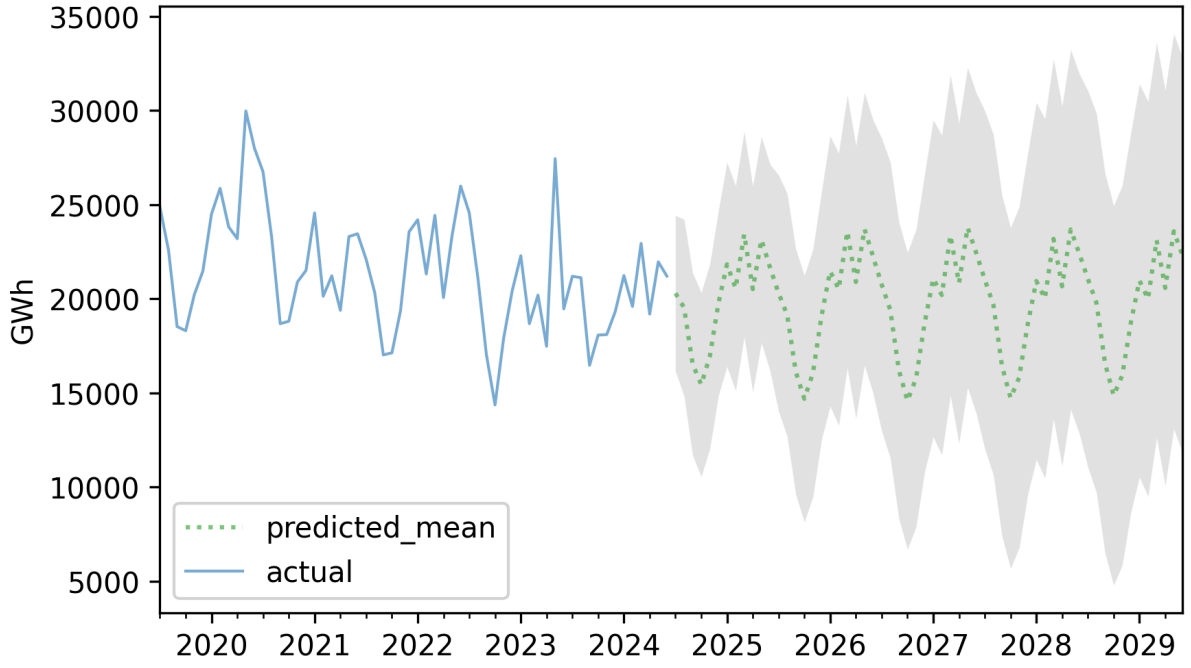



```
# Solution

forecast = results_arima.get_forecast(steps=60)
```

```
# Solution

forecast_mean = forecast.predicted_mean
forecast_ci = forecast.conf_int()
forecast_ci.columns = ["lower", "upper"]
```

Think Stats: Exploratory Data Analysis in Python, 3rd Edition


```
# Solution

recent = hydro.iloc[-60:]

forecast_mean.plot(**pred_options)
plt.fill_between(
    forecast_ci.index,
    forecast_ci.lower,
    forecast_ci.upper,
    lw=0,
    color="gray",
    alpha=0.2,
)
recent.plot(label="actual", **actual_options)
decorate(ylabel="GWh")
```



COMMENTARY

The SARIMA model fitted here, `order=([1, 6], 0, [6])` with `seasonal_order=(0, 1, 0, 12)`, deserves element-by-element unpacking because each number reflects a modelling decision.

The `seasonal_order=(0, 1, 0, 12)` instructs the model to take one round of seasonal differencing at lag 12, forming $\tilde{y}t = y_t - y{t-12}$. This is analogous to year-over-year differencing and removes a seasonal unit root — a pattern where the series in each calendar month follows a random walk rather than reverting to a fixed seasonal level. The differencing stabilises the variance of the seasonal component and is the "I" (integrated) part of the acronym.

The `order=([1, 6], 0, [6])` specifies the non-seasonal AR and MA structure applied to the differenced series. Including lags 1 and 6 in the AR component means the model predicts $\tilde{y}_t$ from the values six months ago and one month ago — plausible for hydroelectric generation, which depends on snowpack and rainfall patterns that carry multi-month memory. The MA lag at 6 models the persistence of forecast errors at a half-year horizon.

The `get_forecast(steps=60)` call uses the fitted state-space representation of the ARIMA model (via a Kalman filter) to propagate the model forward 60 steps. Crucially, the confidence interval widens with the forecast horizon because prediction error variance accumulates: each step adds the one-step-ahead innovation variance $\hat{\sigma}^2$, so the interval grows roughly as $\sqrt{h}$ for an AR(1)-type process. The `conf_int()` method returns the 95% interval by default, derived from the asymptotic normality of the forecast distribution — an assumption that may be too optimistic if the residuals are fat-tailed or if the model is misspecified.

The `n_missing=24` burn-in period reflects a practical constraint: seasonal differencing consumes 12 observations, and the longest non-seasonal lag (6 in both AR and MA) consumes up to 6 more, leaving the first 18 to 24 fitted values unreliable. Trimming them before computing R^2 avoids artificially inflating or deflating the fit statistic due to initialisation artefacts.